Simulation-Based Engineering Lab
University of Wisconsin-Madison

# An Introduction to Computational Fluid Dynamics

Milad Rakhsha, Dan Negrut

Department of Mechanical Engineering, University of Wisconsin – Madison

November 8, 2019

Draft
Milad Rakhsha

# Contents


**1 Notations and Background** **2**
- 1.1 Gradient Operator . . . 2
- 1.2 Divergence Operator . . . 3
- 1.3 Curl Operator . . . 6
- 1.4 Eulerian and Lagrangian Descriptions . . . 7

**2 Conservation Principles** **10**
- 2.1 Fluxes . . . 12
- 2.2 Classifications of Advection-Diffusion-Reaction Equation . . . 14
  - 2.2.1 Parabolic Equations . . . 14
  - 2.2.2 Elliptic Equations . . . 14
  - 2.2.3 Hyperbolic Equations . . . 14
- 2.3 Conservation of Mass and Momentum . . . 15
  - 2.3.1 Mass Conservation . . . 15
  - 2.3.2 Momentum Conservation . . . 15

**3 Numerical Discretization** **19**
- 3.1 Weighted Residual Method . . . 19
- 3.2 Finite Difference Method . . . 21
  - 3.2.1 Boundary Condition Enforcement . . . 22
  - 3.2.2 Conservative vs. non-Conservative Discretization . . . 23
- 3.3 Finite Volume Method . . . 24
  - 3.3.1 First Time Derivative . . . 25
  - 3.3.2 Advective Flux . . . 26
  - 3.3.3 Diffusive Flux . . . 26
  - 3.3.4 Boundary Conditions . . . 27
- 3.4 Finite Element Method . . . 28
  - 3.4.1 Choice of the Weight Function . . . 30
  - 3.4.2 Galerkin Method . . . 30
  - 3.4.3 Enforcement of Dirichlet Condition . . . 32
  - 3.4.4 Calculation of Element and System Matrices . . . 32
  - 3.4.5 Numerical Integration of the Elements' Matrices/Vectors . . . 35
- 3.5 Smoothed Particle Hydrodynamics . . . 41

**4 Numerical Solution of Fluid Flows** **51**
- 4.1 Helmholtz Decomposition . . . 51
- 4.2 Non-Incremental Projection Method . . . 52
- 4.3 Incremental Projection Method . . . 53




Draft
Milad Rakhsha

# 1 Notations and Background

**Zero-rank tensors:** Some physical quantities are described by *scalars*, e.g., density, temperature, and so forth. Scalars are pure numbers, conveying only a sense of magnitude but no sense of direction, although they may have dimensions associated with them. For instance in $\rho = 1000\text{kg/m}^3$, $\rho$ is a scalar with the value of 1000, and $\text{kg/m}^3$ as its unit. We use *non-bold lower-case or non-bold upper-case* letters to denote scalars.
**First-rank tensors:** Some physical quantities are described by *vectors*, e.g., velocity, displacement, and so forth. Vectors imply a sense of direction and magnitude, and they may have dimensions associated with them. For instance in $\boldsymbol{u} = (3, 0, 4)$, $\boldsymbol{u}$ is a vector with the magnitude of $\|\boldsymbol{u}\| = \sqrt{3^2 + 0^2 + 4^2} = 5$ and the unit direction of $\boldsymbol{e_u} = \frac{\boldsymbol{u}}{\|\boldsymbol{u}\|} = (3/5, 0, 4/5)$. We use *bold and lower-case* letters to denote vectors.
**Second-rank tensors:** Some physical quantities are described by *tensors*, e.g., stress, strain, and so forth. Tensors imply a sense of direction and magnitude depending on a direction, and they may have dimensions associated with them just like scalars and vectors. A tensor is defined with 9 elements. For instance,

$$\boldsymbol{\sigma} = \begin{bmatrix} 1 & 0.25 & 0.5 \\ 0.25 & 0.8 & 0.75 \\ 0.5 & 0.75 & 0.7 \end{bmatrix} \tag{1.1}$$

is a second-order tensor that could represent stress at a point. We use *bold and upper-case Latin* letters and usually *bold and lower-case Greek* letters to denote tensors. Unless otherwise noted, the term tensor is used in this manuscript to refer to a second-order tensor.

## 1.1 Gradient Operator

Gradient implies a sense of change in different directions. For instance, by temperature gradient we mean how much the temperature changes if you move in different directions. Mathematically, gradient of a field is the change of that field with respect to change in space. $\partial T/\partial x$ for instance, is a measure of change in $T$ in the x direction. Similarly, $\partial T/\partial y$, and $\partial T/\partial z$ are the amount of change in $T$ in the $y$ and the $z$ directions, respectively. Concisely, we can form a temperature gradient vector as

$$\nabla T = (\frac{\partial T}{\partial x}, \frac{\partial T}{\partial y}, \frac{\partial T}{\partial z}), \tag{1.2}$$

where

$$\nabla = (\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}) \tag{1.3}$$

denotes the gradient operator. Now if are interested in the gradient in any particular direction $\boldsymbol{n} = (n_x, n_y, n_z)$ we can multiple (dot product) $\nabla T$ with the unit vector $n$ as follows,

$$\nabla T\Big|_{\boldsymbol{n}} = \nabla T \cdot \boldsymbol{n} = \frac{\partial T}{\partial x} n_x + \frac{\partial T}{\partial y} n_y + \frac{\partial T}{\partial z} n_z. \tag{1.4}$$



Draft
Milad Rakhsha

For instance, if $\boldsymbol{n} = (1, 0, 0)$ we expect to recover the $\partial T/\partial x$ from the gradient, which is easy to verify from Eq. 1.4.

Note that the gradient of a scalar (zero-order tensor) is a vector (first-order tensor). Similarly, we could think about gradient of a vector. For instance, gradient of a vector such as $\boldsymbol{u} = (u_x, u_y, u_z)$ may be defined as a tensor formed from the gradient of each component of $\boldsymbol{u}$ as follows:

$$\nabla \boldsymbol{u} = \nabla \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} \frac{\partial u_x}{\partial x} & \frac{\partial u_x}{\partial y} & \frac{\partial u_x}{\partial z} \\ \frac{\partial u_y}{\partial x} & \frac{\partial u_y}{\partial y} & \frac{\partial u_y}{\partial z} \\ \frac{\partial u_z}{\partial x} & \frac{\partial u_z}{\partial y} & \frac{\partial u_z}{\partial z} \end{bmatrix}, \tag{1.5}$$

Again, if the gradient in a particular direction $\boldsymbol{n} = (n_x, n_y, n_z)$ we can multiple $\nabla \boldsymbol{u}$ with the unit vector $n$ as follows,

$$\begin{aligned} \nabla \boldsymbol{u}\Big|_{\boldsymbol{n}} = \nabla \boldsymbol{u} \cdot \boldsymbol{n} &= \begin{bmatrix} \frac{\partial u_x}{\partial x} & \frac{\partial u_x}{\partial y} & \frac{\partial u_x}{\partial z} \\ \frac{\partial u_y}{\partial x} & \frac{\partial u_y}{\partial y} & \frac{\partial u_y}{\partial z} \\ \frac{\partial u_z}{\partial x} & \frac{\partial u_z}{\partial y} & \frac{\partial u_z}{\partial z} \end{bmatrix} \begin{bmatrix} n_x \\ n_y \\ n_z \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial u_x}{\partial x} n_x + \frac{\partial u_x}{\partial y} n_y + \frac{\partial u_x}{\partial z} n_z \\ \frac{\partial u_y}{\partial x} n_x + \frac{\partial u_y}{\partial y} n_y + \frac{\partial u_y}{\partial z} n_z \\ \frac{\partial u_z}{\partial x} n_x + \frac{\partial u_z}{\partial y} n_y + \frac{\partial u_z}{\partial z} n_z \end{bmatrix}. \end{aligned} \tag{1.6}$$

For instance, if $\boldsymbol{n} = (1, 0, 0)$ we expect to recover the gradient of $\boldsymbol{u}$ along the $x$ direction, which can be seen from 1.6 to be $[\frac{\partial u_x}{\partial x}, \frac{\partial u_y}{\partial x}, \frac{\partial u_z}{\partial x}]$. This makes a lot of sense because it shows that the gradient of a vector with respect to a component of the coordinate system such as $x$ is the gradient of each component of the vector with respect to $x$.

Lastly, note that the gradient of a vector (first-order tensor) is a second-order tensor. In general, gradient operator *increases* the order of a tensor by 1.

## 1.2 Divergence Operator

Divergence operator provides information about sinks or sources. More precisely, if divergence operator is applied on a **vector** field such as $\boldsymbol{u} = (u_x, u_y, u_z)$ the results is a **scalar** as



Draft
Milad Rakhsha

follows,

$$\begin{aligned}\nabla \cdot \boldsymbol{u} &= (\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}) \cdot (u_x, u_y, u_z) \\ &= \frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} + \frac{\partial u_z}{\partial z}. \end{aligned} \tag{1.7}$$

Now, lets consider three different vector fields shown in Figs 1-3. Divergence of a vector field provide information about generation or dissipation of that field. Where divergence is positive that is an indication of a source and generation of that variable (see Fig 3), while a negative divergence indicates a sink or dissipation (see Fig 2). If divergence of a field at a point is zero, this means there is no sink or source of the variable at that point (see Fig 1).

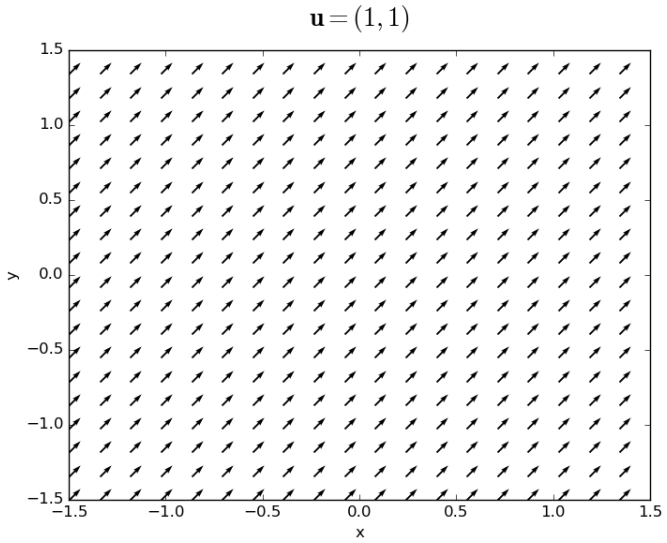



Figure 1: A vector field with no source or sink. The divergence of $\boldsymbol{u}$ at the origin $\nabla \cdot u\big|_{(0,0)} = (\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y})\big|_{(0,0)} = 0$ is zero.



Draft
Milad Rakhsha

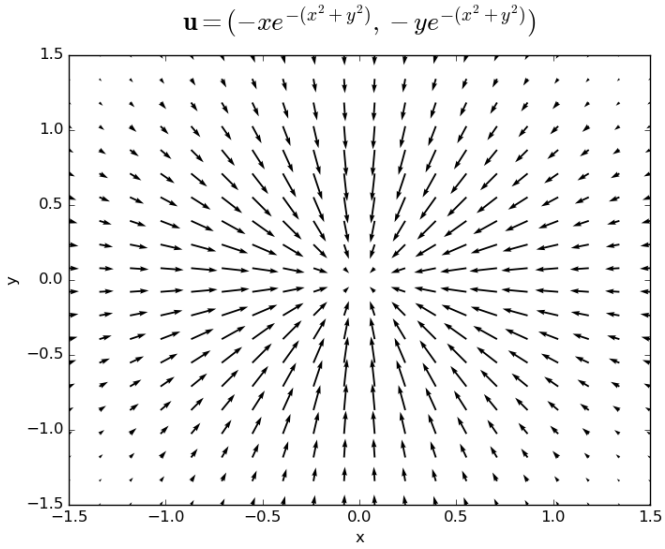



Figure 2: Presence of a sink in a vector field. The divergence of $\boldsymbol{u}$ at the origin $\nabla \cdot u\big|_{(0,0)} = (\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y})\big|_{(0,0)} = -2$ is negative.

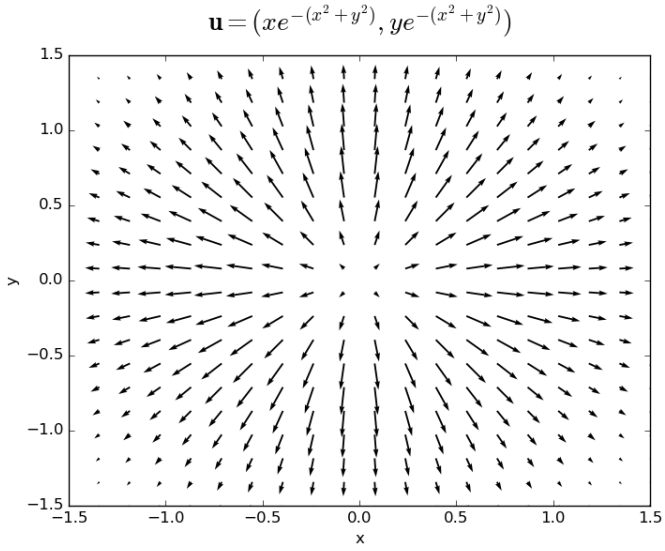



Figure 3: Presence of a source in a vector field. The divergence of $\boldsymbol{u}$ at the origin $\nabla \cdot u\big|_{(0,0)} = (\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y})\big|_{(0,0)} = +2$ is positive.



Draft
Milad Rakhsha

Lastly, divergence of a tensor field $\boldsymbol{S}$ is defined as follows:

$$\nabla \cdot \boldsymbol{S} = [\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}] \cdot \begin{bmatrix} S_{11} & S_{12} & S_{13} \\ S_{21} & S_{22} & S_{23} \\ S_{31} & S_{32} & S_{33} \end{bmatrix} = \begin{bmatrix} \frac{\partial S_{11}}{\partial x} + \frac{\partial S_{12}}{\partial y} + \frac{\partial S_{13}}{\partial z} \\ \frac{\partial S_{21}}{\partial x} + \frac{\partial S_{22}}{\partial y} + \frac{\partial S_{23}}{\partial z} \\ \frac{\partial S_{31}}{\partial x} + \frac{\partial S_{32}}{\partial y} + \frac{\partial S_{33}}{\partial z} \end{bmatrix}. \tag{1.8}$$

Note that divergence operator *decreases* the rank of a tensor field by 1. For instance, the divergence of a vector (first-order tensor) field is a scalar field (zero-order tensor), or the divergence of a tensor (second-order tensor) field is a vector field (first-order tensor).

## 1.3 Curl Operator

The curl operator is defined via $\nabla \times ()$ and it defines the *rotation* vector field of another vector field. More precisely in 3D Cartesian coordinates, the curl of a vector field such as $\boldsymbol{u} = (u_x, u_y, u_z)$ is defined as:

$$\nabla \times \boldsymbol{u} = \begin{vmatrix} \boldsymbol{i} & \boldsymbol{j} & \boldsymbol{k} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ u_x & u_y & u_z \end{vmatrix} = (\frac{\partial u_z}{\partial y} - \frac{\partial u_y}{\partial z})\boldsymbol{i} + (\frac{\partial u_x}{\partial z} - \frac{\partial u_z}{\partial x})\boldsymbol{j} + (\frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y})\boldsymbol{k} \tag{1.9}$$

Subsequently, $\nabla \times \boldsymbol{u} = 0$ denotes an irrotational vector field. An irrotational vector field $\boldsymbol{u}$ is a conservative vector field and may be expressed as the gradient of scalar field, i.e.,

$$\nabla \times \boldsymbol{u} = 0 \Longleftrightarrow \boldsymbol{u} = \nabla \phi. \tag{1.10}$$

In contrast, if a vector field is rotational, i.e., $\nabla \times \boldsymbol{u} \neq 0$ then it is nonconservative and cannot be expressed as the gradient of a scalar field. An example rotational field is shown in Fig. 4



Draft
Milad Rakhsha

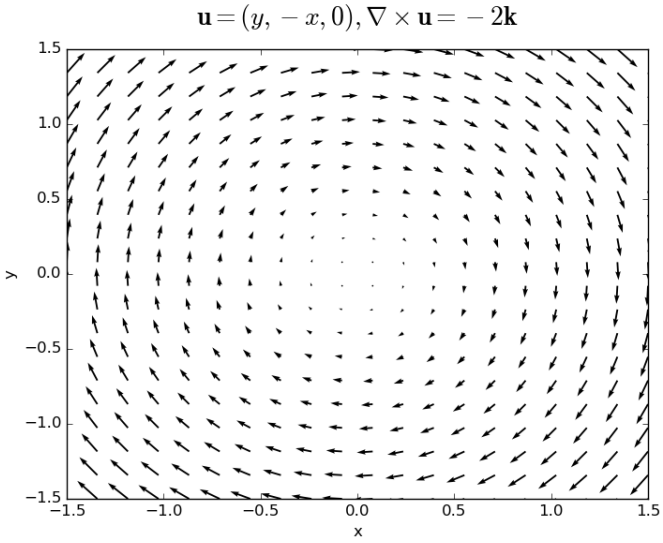



Figure 4: A rotational vector field.

## 1.4 Eulerian and Lagrangian Descriptions

We are interested in calculating the time-derivative of functions of form $f = f(\boldsymbol{x}, t)$. This could be seen from two different perspectives: $(i)$ the time-derivative for a moving observer i.e. when the position is of the observer is time-dependent as $\boldsymbol{x} = \boldsymbol{x}(t)$, and $f = f(\boldsymbol{x}(t), t) = f(x(t), y(t), z(t), t)$ (Lagrangian), and $(ii)$ the time-derivative for an stationary observer (Eulerian). In mathematical words:

$$\begin{aligned}
\frac{Df}{Dt} &= \frac{df(x(t), y(t), z(t), t)}{dt} \\
&= (\frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial x}dx + \frac{\partial f}{\partial y}dy + \frac{\partial f}{\partial z}dz)/dt \\
&= \frac{\partial f}{\partial t} + \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt} + \frac{\partial f}{\partial z}\frac{dz}{dt} \\
&= \frac{\partial f}{\partial t} + \frac{\partial f}{\partial x}u_x + \frac{\partial f}{\partial y}u_y + \frac{\partial f}{\partial z}u_z \\
\underbrace{\frac{Df}{Dt}}_{\text{Material/Lagrangian derivetive}} &= \underbrace{\frac{\partial f}{\partial t}}_{\text{Eulerian derivative}} + \underbrace{\boldsymbol{u} \cdot \nabla f}_{\text{Advective derivative}}
\end{aligned} \tag{1.11}$$

where

$$\boldsymbol{u} = (u_x, u_y, u_z) = (\frac{dx}{dt}, \frac{dy}{dt}, \frac{dz}{dt}). \tag{1.12}$$



Draft
Milad Rakhsha

This is mathematically a simple and obvious fact, but what is the physical meaning of this equation?

For an stationary observer such as a cell in a computational mesh, $\boldsymbol{u} = \mathbf{0}$ and the time-derivative of the function is only related to the change of function w.r.t. time, i.e. $\partial f/\partial t$. For a moving observer such as a moving marker/particle, $\boldsymbol{u} \neq \mathbf{0}$ and the time-derivative has two components: $(i)$ $\partial f/\partial t$ which is the time variation at a fixed point in space, and $(ii)$ $\boldsymbol{u} \cdot \nabla f$, which is the *advective/convective* contribution and comes into play because of the motion of the observer in the field $f$.

Let's look at a steady-state problem where there is a source placed at the origin such that $f = e^{-(x^2+y^2)}$, as shown in Fig. 5.

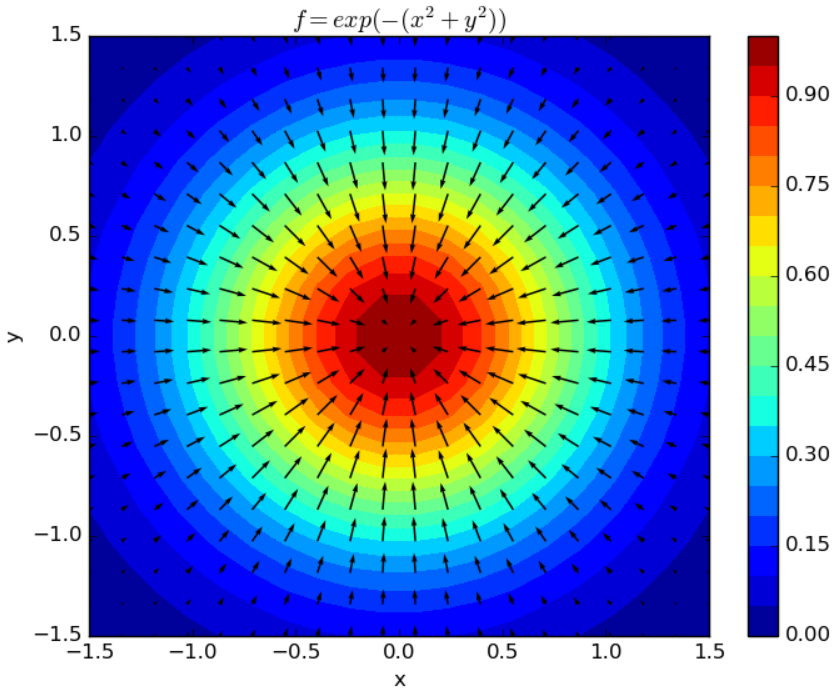



Figure 5: Contour plot of $f = e^{-(x^2+y^2)}$. The gradient of $f$ is shown with arrows.

Let's assume an observer is at $\boldsymbol{x} = (1, 0)$, where the function gradient is $\nabla f = (-0.735, 0)$. If the observer is stationary, it will not sense any time-variation in the $f$ field, owning to the function being time-independent. However, if the observer start moving, depending on the direction of the velocity, it *feels* that $f$ is changing with time. This *advective/convective* rate of change is manifested in the $\boldsymbol{u} \cdot \nabla f$ term. For instance, if $\boldsymbol{u} = (-1, 0)$, then a positive value of $Df/Dt = 0.735$ is obtained, whereas if $\boldsymbol{u} = (+1, 0)$, then a negative value of $Df/Dt = -0.735$ is expected. This makes sense because, one would expect to see a positive change in $f$ w.r.t to time if the observer moves toward the source, and a negative change in $f$



Draft
Milad Rakhsha

w.r.t to time if the observer moves away from it. Interestingly, the advective time-variation is zero ($\boldsymbol{u} \cdot \nabla f = 0$) if the observer moves perpendicular to the function gradient, e.g. if $\boldsymbol{u} = (0, 1)$.

The **Material Derivative** obtained in Eq. 1.11 is applicable to higher-order tensor functions (vector functions, and second-order tensor functions), as well as scalar functions. This is an important concept that frequently appears in conservation principals, and becomes a source of non-linearity in Eulerian frameworks such as Finite-Difference (FD), Finite-Volume (FV), and Finite-Element Methods (FEM). For example, assume that function $f$ is the velocity field itself, then the advective terms that appears in the Navier-Stokes equations becomes $\boldsymbol{u} \cdot \nabla \boldsymbol{u}$ which is a non-linear term when solving for $\boldsymbol{u}$. However, if the *degrees of freedom* are moving markers instead of stationary cells, we can aim for solving $D\boldsymbol{u}/Dt$ directly and not to deal with the non-linearity. Therefore, the type (Lagrangian/Eulerian) of degrees of freedom could lead to a linear (Lagrangian) or a non-linear (Eulerian) problem.



Draft
Milad Rakhsha

# 2 Conservation Principles

In this section we describe the mathematics of the conservation principal, which comes into play when working with variables whose amounts are conserved. For instance, we will be working with *mass, momentum* whose amounts are conserved in given *control volume* ($\boldsymbol{V}$), i.e. these properties can enter and exit the control volume through *control surfaces*($\boldsymbol{S}$), but they cannot emerge out of nothing, see Fig. 6.

Let's assume that $c = c(\boldsymbol{x}, t)$ denotes the concentration of a scalar variable $C$ per unit mass of the control volume, i.e.

$$C = \int_V c(\boldsymbol{x}, t) dm = \int_V c(\boldsymbol{x}, t)\rho(\boldsymbol{x}, t) d\boldsymbol{x} = \int_V \phi(\boldsymbol{x}, t) d\boldsymbol{x}. \tag{2.1}$$

Note that $\phi(\boldsymbol{x}, t) = \rho(\boldsymbol{x}, t)c(\boldsymbol{x}, t)$ is the concentration of $C$ per unit volume, and $\rho = \rho(\boldsymbol{x}, t)$ is the density of the medium. The *flux* of a variable is *the amount of the variable per unit area and unit time* that enters or leaves $V$ through $S$. Accordingly, the rate at which some amount of $C$ enters or leaves the domain with flux $\boldsymbol{f} = \boldsymbol{f}(\boldsymbol{x}, t)$ is given by $\boldsymbol{f} \cdot \boldsymbol{n} ds$ for an infinitesimally small patch $ds$ with a given normal $\boldsymbol{n}$.



Draft
Milad Rakhsha

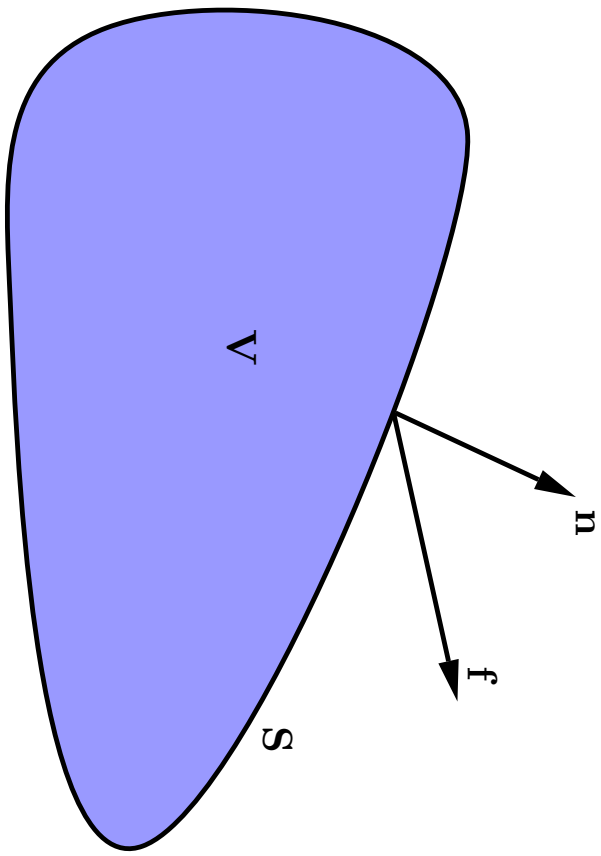



Figure 6: The control volume $\boldsymbol{V}$ bounded by the control surface $\boldsymbol{S}$. At every point on the control surface the vector $\boldsymbol{n}$ is pointing outward, and the $\boldsymbol{f}$ is the flux of a property that enter or exit the control volume.

For a scalar field quantity such as $C$, the flux is a vector field with the same dimension as the dimension of the domain ($V$). For instance, in 2D problems flux of a scalar is a vector in $\mathbb{R}^2$.

We state the conservation principal for any conserved field as follows:

$$\begin{array}{c}\text{the rate of change of}\\ \text{a variable in the}\\ \text{control volume } V\end{array} = \begin{array}{c}\text{the rate at which the}\\ \text{variable enters or}\\ \text{leave the domain}\\ \text{through the control}\\ \text{surface } S\end{array} + \begin{array}{c}\text{the rate of generation}\\ \text{or dissipation of the}\\ \text{variable inside the}\\ \text{control volume } V\end{array} \tag{2.2}$$

or more formally as:

$$\frac{\partial}{\partial t}\int_V \phi(\boldsymbol{x},t)d\boldsymbol{x} = -\int_S \boldsymbol{f}\cdot\boldsymbol{n}ds + \int_V s(\boldsymbol{x},t)d\boldsymbol{x} \tag{2.3}$$



Draft
Milad Rakhsha

where $s(\boldsymbol{x}, t)$ is the *rate* of generation/dissipation of $C$ per unit volume. This is the *integral form* of the conservation law. According to the divergence theorem

$$\int_S \boldsymbol{f} \cdot \boldsymbol{n} ds = \int_V (\nabla \cdot \boldsymbol{f}) d\boldsymbol{x} \tag{2.4}$$

hence, Eq. 2.3 may be written as :

$$\begin{aligned} \frac{\partial}{\partial t} \int_V \phi(\boldsymbol{x}, t) d\boldsymbol{x} &= \int_V (\nabla \cdot \boldsymbol{f}) d\boldsymbol{x} + \int_V s(\boldsymbol{x}, t) d\boldsymbol{x} \\ \int_V \Big[ \frac{\partial \phi(\boldsymbol{x}, t)}{\partial t} &+ \nabla \cdot \boldsymbol{f} - s(\boldsymbol{x}, t) \Big] d\boldsymbol{x} = 0 \end{aligned} \tag{2.5}$$

which is correct for any given control volume. Assuming that the functions $\phi(\boldsymbol{x}, t)$ and $\boldsymbol{f}(\boldsymbol{x}, t)$ are differentiable and the divergence theorem holds, we can obtain the *differential form* of the conservation law as follows,

$$\frac{\partial \phi(\boldsymbol{x}, t)}{\partial t} + \nabla \cdot \boldsymbol{f}(\boldsymbol{x}, t) - s(\boldsymbol{x}, t) = 0 \tag{2.6}$$

Note also that the *differential form* is a stronger statement than the *integral form*, as it is generalized to any point $\boldsymbol{x}$ and any time $t$, while the *integral form* has a a sense of averaging over a volume.

Eq. 2.6 makes sense as it simply states that change in a conserved variable such as $\phi$ may result from the presence of sink/source of $\phi$ or sink/source of the flux of $\phi$ as follows:

$$\underbrace{\frac{\partial \phi(\boldsymbol{x}, t)}{\partial t}}_{\substack{\text{rate of change of } \phi}} = \underbrace{-\nabla \cdot \boldsymbol{f}(\boldsymbol{x}, t)}_{\substack{\text{sink or source of the} \\ \text{flux of } \phi}} + \underbrace{s(\boldsymbol{x}, t)}_{\substack{\text{rate of generation or} \\ \text{dissipation of } \phi}} . \tag{2.7}$$

Above, the negative sing in $-\nabla \cdot \boldsymbol{f}(\boldsymbol{x}, t)$ is due to the convention we chose in Fig. 6 for outward flux. Positive $\nabla \cdot \boldsymbol{f}(\boldsymbol{x}, t)$ means that the control volume is acting as a source of flux (see section §1.2). In other words, positive $\nabla \cdot \boldsymbol{f}(\boldsymbol{x}, t)$ means that $\phi$ is leaving the control volume, $\phi$ should be decreasing. Hence, the sign of $\dfrac{\partial \phi(\boldsymbol{x}, t)}{\partial t}$ should be the opposite of the sign of $\nabla \cdot \boldsymbol{f}(\boldsymbol{x}, t)$.

## 2.1 Fluxes

As mentioned before the flux of a variable $C$ is the amount of $C$ that leaves or enters the domain per unit area and per time. Note that the flux of a scalar (zeroth-order tensor) is a vector (first-order tensor) and the flux of a vector is a tensor (second-order tensor). Advection (convection), and diffusion are two important underlying mechanisms that define the flux. Advective flux of $\phi$ is the transport of $\phi$ through the velocity, i.e. $\phi \boldsymbol{u}$. According



Draft
Milad Rakhsha

to the definition of the flux, the amount of $C$ that passes through the control surface with this flux may be written as $\phi \boldsymbol{u} \cdot \boldsymbol{n} ds dt = \phi(dx/dt) ds dt = \phi dx ds = \phi d\boldsymbol{x}$. On the other hand, diffusion-driven processes are driven by the gradients of concentrations, resulting in $-\mathbf{D}(\boldsymbol{x}, t)\nabla\phi$ type of flux. $\mathbf{D}(\boldsymbol{x}, t)$ is the diffusion coefficient tensor that is simplified to a diagonal tensor for coordinate-independent diffusion processes. Diffusion flux appears in the Navier-Stokes equations from the shear-stress tensor, and in energy equation from the heat conduction, which will be discussed later [1]. Ultimately, the flux vector of $C$ resulting from advection and diffusion of $C$ may be written as follows:

$$\begin{aligned} \boldsymbol{f}(\boldsymbol{x}, t) &= \boldsymbol{u}(\boldsymbol{x}, t)\phi(\boldsymbol{x}, t) - \boldsymbol{D}(\boldsymbol{x}, t)\nabla\phi(\boldsymbol{x}, t) \\ \boldsymbol{f}(\boldsymbol{x}, t) &= \boldsymbol{u}(\boldsymbol{x}, t)\phi(\boldsymbol{x}, t) - \boldsymbol{D}(\boldsymbol{x}, t)\nabla(\rho(\boldsymbol{x}, t)c(\boldsymbol{x}, t)) \end{aligned} \tag{2.8}$$

Substituting the flux in Eq. 2.6 for constant density flow, we get the following form of the transport equation:

$$\frac{\partial\phi(\boldsymbol{x}, t)}{\partial t} + \nabla \cdot (\boldsymbol{u}(\boldsymbol{x}, t)\phi(\boldsymbol{x}, t) - \boldsymbol{D}(\boldsymbol{x}, t)\nabla\phi(\boldsymbol{x}, t)) - s(\boldsymbol{x}, t) = 0. \tag{2.9}$$

Dropping the space and time dependency from Eq. 2.9 for simplicity, the following general form of conservation law is obtained:

$$\frac{\partial\phi}{\partial t} + \nabla \cdot (\boldsymbol{u}\phi - \boldsymbol{D}\nabla\phi) - s = 0 \tag{2.10}$$

Further simplifications to the advective flux is possible assuming an incompressible flow. After incorporating the mass conservation $\nabla \cdot \boldsymbol{u} = 0$ and using the following identity

$$\nabla \cdot (\boldsymbol{u}\phi) = \boldsymbol{u} \cdot \nabla\phi + \phi\nabla \cdot \boldsymbol{u}, \tag{2.11}$$

the form of advection-diffusion-reaction (ADR) equation of a generic transport variable $\phi$ for *incompressible flow* becomes

$$\frac{\partial\phi}{\partial t} + \boldsymbol{u} \cdot \nabla\phi - \nabla \cdot (\boldsymbol{D}\nabla\phi) - s = 0 \tag{2.12}$$

Note that the equivalent Lagrangian form of the equation condenses the advective term and the $\frac{\partial\phi}{\partial t}$ into the material derivative as explained in Eq. 1.11 as follows:

$$\frac{D\phi}{Dt} - \nabla \cdot (\boldsymbol{D}\nabla\phi) - s = 0 \tag{2.13}$$

One last important observation about Eqs. 2.10, and 2.12 is the conservation property of the two formulations. Although the two equations are mathematically equivalent for incompressible flows, applying the same *numerical* discretization to both leads to different results. Eq. 2.10 is the conservative form of the advection-diffusion equation and is used in finite volume method (Section §3.3) while Eq. 2.12 is the non-conservative form and is usually used in finite difference and continuous finite element method (see sections §3.2, 3.4). Using the non-conservative form is usually fine for shock-free flows, whereas the conservative form is used when there is a discontinuity present in the flow. See section §3.2.2 for an example.



Draft
Milad Rakhsha

## 2.2 Classifications of Advection-Diffusion-Reaction Equation

ADR equation is the generic form of many engineering principles; with certain assumption important classes of equations may be constructed from Eq. 2.12 or 2.13. In the following, the details about this classification is explained.

### 2.2.1 Parabolic Equations

The parabolic class of transport equation is obtained when the contribution of the advective term in the equation is negligible. In such as case, the evolution of $\phi$ is driven only by diffusion and reaction. Eq. 2.12 may be simplified as follows:

$$\frac{\partial \phi}{\partial t} - \nabla \cdot (\boldsymbol{D} \nabla \phi) - s = 0 \tag{2.14}$$

This is the typical transport equation when working with unsteady diffusion-driven processes such as mass diffusion or unsteady heat conduction. In mass diffusion equation $\phi = c_i$ is the concentration of the $i$ species, and in head conduction equation $\phi = T$ is the temperature.

### 2.2.2 Elliptic Equations

The elliptic class of transport equation is obtained for steady-state processes where the convection and diffusion and reaction balancing out on another.

$$\boldsymbol{u} \cdot \nabla \phi - \nabla \cdot (\boldsymbol{D} \nabla \phi) - s = 0 \tag{2.15}$$

This is the typical transport equation when working with steady-state systems, e.g., porous media, steady-state temperature distribution. Negligible advection allows for further simplification to Poisson-like equations as follows:

$$-\nabla \cdot (\boldsymbol{D} \nabla \phi) - s = 0 \tag{2.16}$$

If the diffusion coefficient is symmetric and diagonal, Eq. 2.16 leads to $\nabla^2 \phi = s$, which comes into play when solving the pressure equation in the Navier-Stokes equations.

### 2.2.3 Hyperbolic Equations

If the diffusion has negligible effect, the ADR equation leads to hyperbolic class of problems as follows:

$$\frac{\partial \phi}{\partial t} + \boldsymbol{u} \cdot \nabla \phi - s = 0 \tag{2.17}$$

The mass conservation, is an example hyperbolic equation that will be discussed later by replacing $\phi = \rho$ and assuming that there is no mass sink or source in the field, i.e., $s = 0$. Another prominent example is the level-set equation which is obtained if $\phi$ is the level-set function.



Draft
Milad Rakhsha

## 2.3 Conservation of Mass and Momentum

We can derive the mass and momentum balance from the conservation law explained in Eq. 2.6, by specifying the conserved quantity $C$ to be either the mass or momentum concentration.

### 2.3.1 Mass Conservation

Assume that $c = 1$, i.e. $C$ is a *scalar* specifying the concentration of mass, and the integral in Eq. 2.1 is the physical *mass* of the control volume. Substituting the $\phi = \rho c = \rho$ in Eq. 2.6 and assuming the advective flux vector of $\boldsymbol{f} = \boldsymbol{u}\phi = \rho\boldsymbol{u}$ and no mass source ($s = 0$) leads to the generic forms of the continuity equation:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \boldsymbol{u}) = 0, \tag{2.18a}$$

$$\frac{\partial \rho}{\partial t} + \boldsymbol{u} \cdot \nabla \rho + \rho \nabla \cdot \boldsymbol{u} = 0, \tag{2.18b}$$

$$\frac{D\rho}{Dt} + \rho \nabla \cdot \boldsymbol{u} = 0. \tag{2.18c}$$

Note that Eq. 2.18c is derived from Eq. 2.18b condensing the first two terms into the material derivative as explained in Eq. 1.11. Incompressible flow assumption ($D\rho/Dt = 0$) may further simplify the mass conservation equations Eqs. 2.18 to:

$$\nabla \cdot \boldsymbol{u} = 0. \tag{2.19}$$

Note that $\nabla \cdot \boldsymbol{u} = 0$ is **not** the incompressible flow assumption, but the consequence of this assumption in the mass conservation.

### 2.3.2 Momentum Conservation

Similarly, we can derive the conservation of momentum, also called Navier-Stokes equations, assuming that $c = \mathbf{u}$ and $\phi = \rho c = \rho\mathbf{u}$, i.e. $C$ is a *vector* specifying the momentum concentration. As mentioned previously, the flux of a vector is a second-order tensor. We can write the flux tensor of the momentum as the sum of advective and stress flux, i.e.

$$\begin{aligned} \boldsymbol{f} =& \boldsymbol{u}(\boldsymbol{x}, t)\phi(\boldsymbol{x}, t) - \boldsymbol{\sigma}(\boldsymbol{x}, t) = \rho\boldsymbol{u}\boldsymbol{u} - \boldsymbol{\sigma} \\ =& \rho \begin{bmatrix} u_1u_1 & u_1u_2 & u_1u_3 \\ u_2u_1 & u_2u_2 & u_2u_3 \\ u_3u_1 & u_3u_2 & u_3u_3 \end{bmatrix} + \begin{bmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_{22} & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix} \end{aligned} \tag{2.20}$$

where $\boldsymbol{u}\boldsymbol{u}$ is a tensor results from the dyadic product of $\boldsymbol{u}$ by itself, and $\boldsymbol{\sigma}(\boldsymbol{x}, t)$ is the stress flux. $\boldsymbol{\sigma}$ can be decomposed into a volumetric (hydrostatic) part and a *traceless* deviatoric



Draft
Milad Rakhsha

part, as follows:

$$\boldsymbol{\sigma} = \boldsymbol{\sigma}_{dev} + \boldsymbol{\sigma}_{vol} \tag{2.21}$$
$$tr(\boldsymbol{\sigma}_{vol}) = tr(\boldsymbol{\sigma}) = \sigma_{11} + \sigma_{22} + \sigma_{33}, \tag{2.22}$$
$$tr(\boldsymbol{\sigma}_{dev}) = 0. \tag{2.23}$$

where $tr()$ indicates sum of the diagonal elements of a tensor. Defining

$$p = -(\sigma_{11} + \sigma_{22} + \sigma_{33})/3, \tag{2.24}$$

to be the mechanical pressure, we can write the volumetric and the deviatoric parts of the stress tensor as follows:

$$\boldsymbol{\sigma}_{vol} = (\frac{\boldsymbol{\sigma} : \mathbf{I}}{3})\mathbf{I} = (\frac{\sigma_{11} + \sigma_{22} + \sigma_{33}}{3})\mathbf{I} = -p \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{2.25}$$

$$\boldsymbol{\sigma}_{dev} = \boldsymbol{\sigma} - (\frac{\boldsymbol{\sigma} : \mathbf{I}}{3})\mathbf{I} = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_{22} & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix} + \begin{bmatrix} p & 0 & 0 \\ 0 & p & 0 \\ 0 & 0 & p \end{bmatrix} \tag{2.26}$$

Ultimately, substituting the flux of Eq. 2.20 and stress decomposition of Eqs. 2.21, 2.25 and 2.26 into Eq. 2.6 leads to:

$$\frac{\partial \rho \boldsymbol{u}}{\partial t} + \nabla \cdot (\rho \boldsymbol{u}\boldsymbol{u} - \boldsymbol{\sigma}_{vol} - \boldsymbol{\sigma}_{dev}) - \boldsymbol{f}_b = 0, \tag{2.27}$$

where $\boldsymbol{f}_b$ is the body force that acts as the source term $\boldsymbol{s}$ seen in Eqs. 2.12-2.13. Applying the identity in Eq. 2.11, and applying the mass conservation for incompressible flow (Eq. 2.19) the second term can be simplified to

$$\nabla \cdot (\rho \boldsymbol{u}\boldsymbol{u}) = \boldsymbol{u} \cdot \nabla(\rho \boldsymbol{u}) + \rho \boldsymbol{u}(\nabla \cdot \boldsymbol{u}) = \boldsymbol{u} \cdot \nabla(\rho \boldsymbol{u}). \tag{2.28}$$

Considering the identity discussed in Eq. 1.8 the third term may be written as :

$$\nabla \cdot (-\boldsymbol{\sigma}_{vol}) = \nabla \cdot (p\mathbf{I}) = \nabla p \tag{2.29}$$

To summarize, the momentum conservation becomes:

$$\frac{\partial \rho \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla(\rho \boldsymbol{u}) + \nabla p - \nabla \cdot (\boldsymbol{\sigma}_{dev}) - \boldsymbol{f}_b = 0. \tag{2.30}$$

Up to this point, the only assumption was the incompressibility of the medium ($D\rho/Dt = 0$). Hence, the momentum equations are applicable to any material satisfying this condition. The modeling of the last term needs a constitutive equation, which depends on the material properties. For Newtonian fluids, the deviatoric part of the stress tensor is related to the



Draft
Milad Rakhsha

shear rate, through $\boldsymbol{\sigma}_{dev} = 2\mu\boldsymbol{E}$, where $\mu$ is a constant representing the dynamic viscosity of the fluid and shear rate tensor $\boldsymbol{E}$ is the symmetric part of the velocity gradient tensor, i.e.

$$\boldsymbol{E} = \frac{1}{2}(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^T). \tag{2.31}$$

Given the velocity gradient tensor,

$$\nabla \boldsymbol{u} = \nabla \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} \frac{\partial u_x}{\partial x} & \frac{\partial u_x}{\partial y} & \frac{\partial u_x}{\partial z} \\ \frac{\partial u_y}{\partial x} & \frac{\partial u_y}{\partial y} & \frac{\partial u_y}{\partial z} \\ \frac{\partial u_z}{\partial x} & \frac{\partial u_z}{\partial y} & \frac{\partial u_z}{\partial z} \end{bmatrix}, \tag{2.32}$$

$$\nabla \boldsymbol{u}^T = \begin{bmatrix} \frac{\partial u_x}{\partial x} & \frac{\partial u_y}{\partial x} & \frac{\partial u_z}{\partial x} \\ \frac{\partial u_x}{\partial y} & \frac{\partial u_y}{\partial y} & \frac{\partial u_z}{\partial y} \\ \frac{\partial u_x}{\partial z} & \frac{\partial u_z}{\partial z} & \frac{\partial u_z}{\partial z} \end{bmatrix}, \tag{2.33}$$

(2.34)

and the relation given in Eq. 1.8 the $\nabla \cdot (\boldsymbol{\sigma}_{dev}) = \nabla \cdot (\mu(\nabla \mathbf{u} + \nabla \mathbf{u}^{\mathbf{T}}))$ term may be simplified considering that

$$\nabla \cdot (\nabla \boldsymbol{u}) = \begin{bmatrix} \frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_x}{\partial y^2} + \frac{\partial^2 u_x}{\partial z^2} \\ \frac{\partial^2 u_y}{\partial x^2} + \frac{\partial^2 u_y}{\partial y^2} + \frac{\partial^2 u_y}{\partial z^2} \\ \frac{\partial^2 u_z}{\partial x^2} + \frac{\partial^2 u_z}{\partial y^2} + \frac{\partial^2 u_z}{\partial z^2} \end{bmatrix} = \nabla^2(\boldsymbol{u}), \tag{2.35}$$

$$\nabla \cdot (\nabla \boldsymbol{u}^T) = \begin{bmatrix} \frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_y}{\partial x \partial y} + \frac{\partial^2 u_z}{\partial x \partial z} \\ \frac{\partial^2 u_x}{\partial x \partial y} + \frac{\partial^2 u_y}{\partial y^2} + \frac{\partial^2 u_z}{\partial y \partial z} \\ \frac{\partial^2 u_x}{\partial x \partial z} + \frac{\partial^2 u_z}{\partial y \partial z} + \frac{\partial^2 u_z}{\partial z^2} \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x}(\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} + \frac{\partial u_z}{\partial z}) \\ \frac{\partial}{\partial y}(\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} + \frac{\partial u_z}{\partial z}) \\ \frac{\partial}{\partial z}(\frac{\partial u_x}{\partial x} + \frac{\partial u_z}{\partial y} + \frac{\partial u_z}{\partial z}) \end{bmatrix} = \nabla(\nabla \cdot \boldsymbol{u}) = \mathbf{0}. \tag{2.36}$$

where $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}$ is the Laplacian operator, to

$$\nabla.(\boldsymbol{\sigma}_{dev}) = \nabla.(2\mu\boldsymbol{E}) = \nabla.(\mu(\nabla \mathbf{u} + \nabla \mathbf{u}^{\mathbf{T}})) = \mu\nabla^{\mathbf{2}}\mathbf{u}, \tag{2.37}$$



Draft
Milad Rakhsha

The final form of the momentum conservation described in Eq. 2.30 under incompressible flow assumption and Newtonian fluid material is as follows:

$$\frac{\partial \rho \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla(\rho \boldsymbol{u}) + \nabla p - \mu \nabla^2 \boldsymbol{u} - \boldsymbol{f}_b = 0, \tag{2.38a}$$

$$\frac{D(\rho \boldsymbol{u})}{Dt} + \nabla p - \mu \nabla^2 \boldsymbol{u} - \boldsymbol{f}_b = 0, \tag{2.38b}$$

where Eq. 2.38a is used in Eulerian frameworks and Eq. 2.38b is the Lagrangian counterpart obtained after condensing the first two terms of Eq. 2.38a via the material derivative relation (Eq. 1.11).



Draft
Milad Rakhsha

# 3 Numerical Discretization

## 3.1 Weighted Residual Method

In this section we introduce the weighted residual method which is closely related to the weak form of the conservation laws. This is important in the numerical procedure which will be explained later for broadening the class of admissible functions that satisfy the conservation laws only in an *integral/weak* sense [1].

Let's assume that $\bar{\boldsymbol{\phi}}$ is an *approximate* solution to the advection-diffusion equation discussed in Eq. 2.12 in a domain $V$ bounded by the boundary $S$, similar to the one shown in Figure 6. Let's assume that on some portion of the boundary $S_D \subset S$, the value of $\phi$ is known at all times, i.e. $\phi(\boldsymbol{x}, t) = \phi_D(\boldsymbol{x})$, $\forall \boldsymbol{x} \in S_D$, and $\forall t \in (0, T)$. This sort of boundary conditions are known as *Dirichlet* boundary conditions. On the remaining part of the boundary $S_N = S \setminus S_D$ the value of flux is known, i.e. $\boldsymbol{f}(\boldsymbol{x}, t) \cdot \boldsymbol{n} = g(\boldsymbol{x})$, $\forall \boldsymbol{x} \in S_N$, and $\forall t \in (0, T)$. The condition is known as *Neumann* boundary condition. The flux in the Neumann condition is usually either the diffusive flux, or sum of the advective and diffusive flux. Note that if only the advective flux $\boldsymbol{u}\phi$ is known, this condition is similar to the Dirichlet condition. The residual of the numerical approximation may be defined as follows:

$$\mathcal{R}(\bar{\phi}) = \frac{\partial \bar{\phi}}{\partial t} + \nabla \cdot (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) - s. \tag{3.1}$$

Clearly, if $\bar{\phi}$ is the exact solution then the $\mathcal{R}(\bar{\phi}) = 0$. Hence, the magnitude of $\mathcal{R}(\bar{\phi})$ is a measure of accuracy of the solution. In the finite element method, we this residual equation on nodal points. Now assume that we would like to satisfy this equation but with a weight that dictates higher accuracy at some points compared to other points. For example, if this weight is one at the node $i$ and goes to zero away from the node $i$ we have *weighted* the *residual* equation for nodal point $i$. More formally we do so by multiplying this residual function by a suitably chosen *weight* or *test* function $w$ that vanishes on the *Dirichlet* part of the boundary, i.e. $w = 0$ on $S_D$. We shall explain in more details about ways in which this test/weight function can be selected in section §3.4.1. See [2] for a more mathematically rigorous description. After integrating over the domain it follows that

$$\int_V w\mathcal{R}(\bar{\phi}) d\boldsymbol{x} = 0, \qquad \forall w \in \mathcal{W} \tag{3.2a}$$

$$\int_V w\Big(\frac{\partial \bar{\phi}}{\partial t} + \nabla \cdot (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) - s\Big) d\boldsymbol{x} = 0, \qquad \forall w \in \mathcal{W} \tag{3.2b}$$

Applying integration by parts on the flux term, and incorporating the divergence theorem



Draft
Milad Rakhsha

in Eq. 2.4 results in:

$$\int_V w\nabla\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})d\boldsymbol{x}=\int_V\nabla\cdot\big(w(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})\big)d\boldsymbol{x}-\int_V\nabla w\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})d\boldsymbol{x} \tag{3.3}$$

$$\int_V w\nabla\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})=\int_S w(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds-\int_V\nabla w\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})d\boldsymbol{x} \tag{3.4}$$

$$\int_V w\nabla\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})=\int_{S_N} w(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds-\int_V\nabla w\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})d\boldsymbol{x} \tag{3.5}$$

Substituting Eq. 3.5 into the flux term of the Eq. 3.2b leads to:

$$\int_V(w\frac{\partial\bar{\phi}}{\partial t}-\nabla w\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})-ws)d\boldsymbol{x}+\int_{S_N} w(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds=0,\qquad\forall w\in\mathcal{W} \tag{3.6}$$

incorporating the Neumann boundary condition $\boldsymbol{f}(\boldsymbol{x},t)\cdot\boldsymbol{n}=(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}=g(\boldsymbol{x}),\ \forall\boldsymbol{x}\in S_N$, and $\forall t\in(0,T)$ we obtain the weak form as follows:

$$\int_V(w\frac{\partial\bar{\phi}}{\partial t}-\nabla w\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})-ws)d\boldsymbol{x}+\int_{S_N} w\,g\,ds=0,\qquad\forall w\in\mathcal{W} \tag{3.7}$$

The weak form of the conservation law is elegant as it allows the recovery of both the integral and differential forms of the conservation law. It is easy to recover the integral form of the conservation law by setting $w=1$ in Eq. 3.6 which results in

$$\int_V\frac{\partial\bar{\phi}}{\partial t}d\boldsymbol{x}+\int_{S_N}(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds=\int_V sd\boldsymbol{x}, \tag{3.8}$$

as described in Eq. 2.3. To recover the strong form at a *collocation* point $\boldsymbol{x}_0\in V$, replace $w(\boldsymbol{x})=\delta(\boldsymbol{x}-\boldsymbol{x}_0)$ in Eq. 3.2b and use the identity $\int f\delta(\boldsymbol{x}-\boldsymbol{x}_0)dV=f(\boldsymbol{x}_0)$, which results in

$$\frac{\partial\bar{\phi}}{\partial t}+\nabla\cdot(\boldsymbol{u}\bar{\phi}-\boldsymbol{D}\nabla\bar{\phi})=s\qquad\text{at }\boldsymbol{x}=\boldsymbol{x}_0 \tag{3.9}$$



Draft
Milad Rakhsha

## 3.2 Finite Difference Method

Finite difference method is probably the oldest discretization technique that is used for solving partial differential equations. The idea is to discretize the underlying equations using Taylor expansion on a grid that defines the degrees of freedom as shown Figure 7.

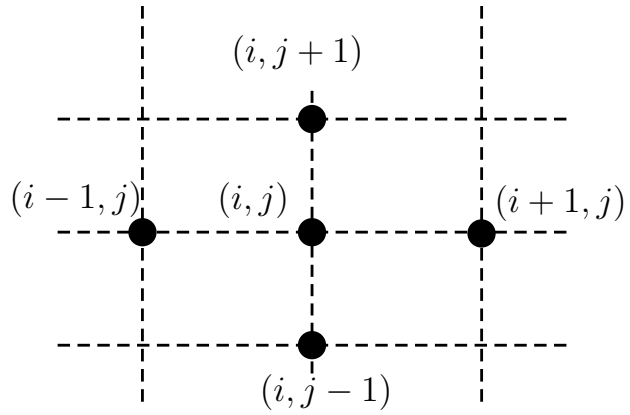



Figure 7: The discretization grid around the degree of freedom $(i, j)$ and its neighbors.

Herein, the degrees of freedoms are defined at **vertices** as (for 2D problems)

$$\phi_{(i,j)}(t) = \phi(\boldsymbol{x}_{(i,j)}, t). \tag{3.10}$$

Subsequently, according to the Taylor expansion of $\phi(\boldsymbol{x}, t)$ around point $(i, j)$, the following partial derivatives are obtained:

$$\frac{\partial \phi}{\partial x}|_{(i,j)} = \frac{\phi_{(i+1,j)}(t) - \phi_{(i-1,j)}(t)}{2\Delta x} \tag{3.11}$$

$$\frac{\partial \phi}{\partial y}|_{(i,j)} = \frac{\phi_{(i,j+1)}(t) - \phi_{(i,j-1)}(t)}{2\Delta y} \tag{3.12}$$

$$\frac{\partial^2 \phi}{\partial x^2}|_{(i,j)} = \frac{\phi_{(i+1,j)}(t) - 2\phi_{(i,j)}(t) + \phi_{(i-1,j)}(t)}{\Delta x^2} \tag{3.13}$$

$$\frac{\partial^2 \phi}{\partial y^2}|_{(i,j)} = \frac{\phi_{(i,j+1)}(t) - 2\phi_{(i,j)}(t) + \phi_{(i,j-1)}(t)}{\Delta y^2} \tag{3.14}$$

The time derivative terms can be computed similar to finite volume method in Eq. 3.35 as

$$\frac{\partial \phi_{(i,j)}(t)}{\partial t} = \frac{\phi_{(i,j)}(t^n) - \phi_{(i,j)}(t^{n-1})}{\Delta t} \tag{3.15}$$

or via a second-order accurate scheme as follows:

$$\frac{\partial \phi_{(i,j)}(t)}{\partial t} = \frac{3\phi_{(i,j)}(t^n) - 4\phi_{(i,j)}(t^{n-1}) + \phi_{(i,j)}(t^{n-2})}{2\Delta t} \tag{3.16}$$



Draft
Milad Rakhsha

Finally, the strong form of the conservation law stated in Eq. 2.12, for a diagonal diffusion tensor, is modified as follows:

$$\frac{\partial \phi}{\partial t} + \boldsymbol{u} \cdot \nabla \phi - D\nabla^2 \phi - s = 0 \tag{3.17}$$

Note that $\nabla^2 \phi = \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2}$, in 2D Cartesian domain. Using Eqs. 3.11-3.14 Eq. 3.17 is discretized as :

$$\begin{aligned} &\frac{\phi_{(i,j)}(t^n) - \phi_{(i,j)}(t^{n-1})}{\Delta t} + u_x \frac{\phi_{(i+1,j)}(t) - \phi_{(i-1,j)}(t)}{2\Delta x} + u_y \frac{\phi_{(i,j+1)}(t) - \phi_{(i,j-1)}(t)}{2\Delta y} \\ &- D\Big(\frac{\phi_{(i+1,j)}(t) - 2\phi_{(i,j)}(t) + \phi_{(i-1,j)}(t)}{\Delta x^2} + \frac{\phi_{(i,j+1)}(t) - 2\phi_{(i,j)}(t) + \phi_{(i,j-1)}(t)}{\Delta y^2}\Big) \\ &- s_{(i,j)} = 0 \end{aligned} \tag{3.18}$$

which is the algebraic equation for every internal node $\boldsymbol{x}_{(i,j)}$.

### 3.2.1 Boundary Condition Enforcement

In regard to Dirichlet boundary conditions, the algebraic equation arises from the discretization of the strong form of the transport equation 2.12 is replaced with $\phi_{(i,j)}(t) = \phi_D(\boldsymbol{x}_{(i,j)})$ on any node that is on the Dirichlet part of the boundary ($S_D$). Pure Neumann boundary condition ($(\nabla.\phi) \cdot \boldsymbol{n}$) may be applied via Eq. 3.11 or Eq. 3.12 depending on the direction of the normal vector. For instance, on a boundary point on the left edge and right edge of the domain, respectively the following holds :

$$(\nabla \phi) \cdot \boldsymbol{n} = h = \frac{3\phi_{(i,j)}(t) - 4\phi_{(i+1,j)}(t) + \phi_{(i+2,j)}(t)}{2\Delta x}, \tag{3.19}$$

$$(\nabla \phi) \cdot \boldsymbol{n} = h = \frac{3\phi_{(i,j)}(t) - 4\phi_{(i-1,j)}(t) + \phi_{(i-2,j)}(t)}{2\Delta x}. \tag{3.20}$$

We can make the same argument about the top and bottom edges. Finally, flux type of boundary condition may be posed as a Robin boundary condition as follows:

$$(\boldsymbol{u}\phi - \boldsymbol{D}\nabla\phi) \cdot \boldsymbol{n} = h = (\boldsymbol{u} \cdot \boldsymbol{n})\phi - (D\nabla\phi \cdot \boldsymbol{n}) \tag{3.21}$$

For instance for a point on the left edge of the domain,

$$(\boldsymbol{u}_{(i,j)} \cdot (-1, 0))\phi_{(i,j)} - D(\frac{3\phi_{(i,j)}(t) - 4\phi_{(i+1,j)}(t) + \phi_{(i+2,j)}(t)}{2\Delta x}) = h \tag{3.22}$$

is the algebraic equation that needs to be solved.



Draft
Milad Rakhsha

### 3.2.2 Conservative vs. non-Conservative Discretization

We now turn our attention to the discretization of the advection term. The non-conservative form of the advection term is $\boldsymbol{u} \cdot \nabla\phi + \phi\nabla \cdot \boldsymbol{u}$ (see Eqs. 2.11-2.12), whereas the corresponding conservative form $\nabla \cdot (\boldsymbol{u}\phi)$ discussed in Eq. 2.10. As explained in section §2.1, the two formulations are not numerically equivalent. The 1D ($\nabla = \partial/\partial x$, $\boldsymbol{u} = u_x$) finite difference discretization of the conservative form for an internal node is as follow:

$$\frac{\partial(u_x\phi)}{\partial x} \approx \frac{u_{x_{i+1}}\phi_{i+1} - u_{x_{i-1}}\phi_{i-1}}{2\Delta x}. \tag{3.23}$$

The discretization of the non-conservative form without assuming the divergence free condition $\nabla \cdot \boldsymbol{u} = \frac{\partial u}{\partial x} = 0$ , however, is as follows:

$$u\frac{\partial\phi}{\partial x} + \phi\frac{\partial u_x}{\partial x} \approx u_{x_i}\frac{\phi_{i+1} - \phi_{i-1}}{2\Delta x} + \phi_i\frac{u_{x_{i+1}} - u_{x_{i-1}}}{2\Delta x}. \tag{3.24}$$

As seen, the two discretization are not similar, while the original derivative is mathematically the same. The 2D ($\nabla = (\partial/\partial x, \partial/\partial y)$, $\boldsymbol{u} = (u_x, u_y)$) finite difference discretization of the conservative form $\nabla \cdot (\boldsymbol{u}\phi)$ for an internal node is as follow:

$$\begin{aligned}\nabla \cdot (\boldsymbol{u}\phi) &= \frac{\partial(u_x\phi)}{\partial x} + \frac{\partial(u_y\phi)}{\partial y} \\ &\approx \frac{u_{x_{i+1,j}}\phi_{i+1,j} - u_{x_{i-1,j}}\phi_{i-1,j}}{2\Delta x} + \frac{u_{y_{i,j+1}}\phi_{i,j+1} - u_{y_{i,j-1}}\phi_{i,j-1}}{2\Delta y}\end{aligned} \tag{3.25}$$

The discretization of the non-conservative form $\boldsymbol{u} \cdot \nabla\phi + \phi\nabla \cdot \boldsymbol{u}$ without assuming the divergence free condition $\nabla \cdot \boldsymbol{u} = \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} = 0$ , however, is as follows:

$$\begin{aligned}\boldsymbol{u} \cdot \nabla\phi + \phi\nabla \cdot \boldsymbol{u} =& (u_x\frac{\partial\phi}{\partial x} + u_y\frac{\partial\phi}{\partial y}) + (\phi\frac{\partial u_x}{\partial x} + \phi\frac{\partial u_y}{\partial y}) \\ &\approx u_{x_{i,j}}\frac{\phi_{i+1,j} - \phi_{i-1,j}}{2\Delta x} + u_{y_{i,j}}\frac{\phi_{i,j+1} - \phi_{i,j-1}}{2\Delta y} \\ &+ \phi_{i,j}(\frac{u_{x_{i+1,j}} - u_{x_{i-1,j}}}{2\Delta x} + \frac{u_{x_{i,j+1}} - u_{x_{i,j-1}}}{2\Delta y}).\end{aligned} \tag{3.26}$$

Again, we see that although the two formulations are mathematically the same the numerical discretizations are different. In the next section we will show that finite volume is built upon the integral, and conservative form of the equations, ensuring of conservation principals, while FD and FE are built from the non-conservative form of the equations.



Draft
Milad Rakhsha

## 3.3 Finite Volume Method

The idea of finite volume is simple; it solves the integral form of the conservation laws on discrete volumes that make up a mesh. In other words, for each cell in the computational grid, the integral form of the equations are solved. For instance, consider the conservation law in Eq. 2.3 for a cell $V_i$ bounded by the surface $S_i$. The integral form of the conservation law is

$$\frac{\partial}{\partial t}\int_{V_i}\phi(\boldsymbol{x},t)d\boldsymbol{x} = -\int_{S_i}\boldsymbol{f}\cdot\boldsymbol{n}ds + \int_{V_i}s(\boldsymbol{x},t)d\boldsymbol{x}. \tag{3.27}$$

Now assume that the value of $\phi_i$ (also the degree of freedom in cell $i$) represents an average value of $\phi$ in $V_i$, i.e.

$$\phi_i(t) = \frac{1}{|V_i|}\int_{V_i}\phi(\boldsymbol{x},t)dVi... \tag{3.28}$$

Similarly, the source term in the cell can be approximated as the average value of the cell, i.e.

$$s_i(t) = \frac{1}{|V_i|}\int_{V_i}s(\boldsymbol{x},t)dV. \tag{3.29}$$

Also assume that the cell $V_i$ is a polyhedral which can be represent by a volume surrounded by flat faces with known surface area and surface normal as shown in Figure 8.

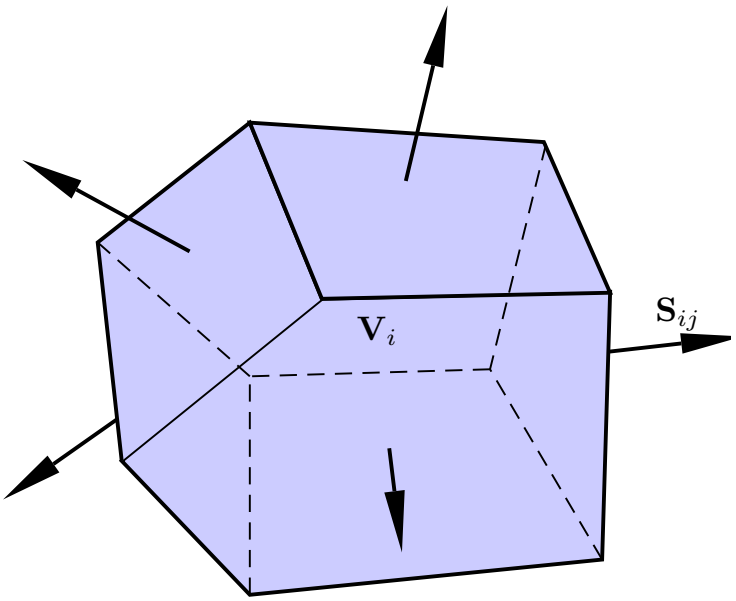



Figure 8: The cell $V_i$ bounded by the control surface $S_i$. The control surface $S_i$ consists of flat surfaces such as $\boldsymbol{S}_{ij}$ with known surface area $S_{ij}$ and surface normal vector $\boldsymbol{n}_{ij}$ such that $\boldsymbol{S}_{ij} = S_{ij}\,\boldsymbol{n}_{ij}$



Draft
Milad Rakhsha

This assumption allows the the surface integral in Eq. 3.27 to be written as summation over the neighboring faces of the cell $V_i$, i.e.

$$\int_{S_i} \boldsymbol{f} \cdot \boldsymbol{n} ds = \sum_{S_{ij}} \boldsymbol{f} \cdot \boldsymbol{S_{ij}} \tag{3.30}$$

Substituting Eqs. 3.28-3.30 into Eq. 3.27 leads to the discrete form of the equation on cell $i$ as follows:

$$\frac{\partial}{\partial t} \int_{V_i} \phi(\boldsymbol{x}, t) d\boldsymbol{x} = - \int_{S_i} \boldsymbol{f} \cdot \boldsymbol{n} ds + \int_{V_i} s(\boldsymbol{x}, t) d\boldsymbol{x} \tag{3.31}$$

$$|V_i| \frac{\partial \phi_i(t)}{\partial t} = - \sum_{S_{ij}} \boldsymbol{f} \cdot \boldsymbol{S_{ij}} + |V_i| s_i(t) \tag{3.32}$$

$$|V_i| \frac{\partial \phi_i(t)}{\partial t} = - \sum_{S_{ij}} (\boldsymbol{u}\phi - \boldsymbol{D}\nabla\phi) \cdot \boldsymbol{S_{ij}} + |V_i| s_i(t) \tag{3.33}$$

$$|V_i| \frac{\partial \phi_i(t)}{\partial t} = - \underbrace{\sum_{S_{ij}} (\boldsymbol{u}\phi) \cdot \boldsymbol{S_{ij}}}_{\text{advctive flux}} + \underbrace{\sum_{S_{ij}} (\boldsymbol{D}\nabla\phi) \cdot \boldsymbol{S_{ij}}}_{\text{diffusive flux}} + |V_i| s_i(t) \tag{3.34}$$

Above, note that the discretization is built upon the integral form of the conservation laws (Eq. 2.5), while maintaining the conservative form of the advection term (see section §3.2.2). Hence, it is often said that Finite Volume method is both locally and globally conservative.

In the following section we describe how to calculate each term in Eq. 3.34 as a function of the degrees of freedom, i.e. cell center values such as $\phi_i$, $\phi_j$. At the end, it will be clear how this conservation principal can be written for cell $i$. Performing the same operation over all of the cells in a domain, we can come up with $n$ equations for $n$ cells, and solve a linear system of equations to solve for $\phi_i$ values.

### 3.3.1 First Time Derivative

The first time derivative of $\phi_i(t)$ in Eq. 3.34 may be approximated in various ways including :

- **Euler forward :** this first-order accurate (in time) scheme is as follows:

$$\frac{\partial \phi_i(t)}{\partial t} = \frac{\phi_i(t^n) - \phi_i(t^{n-1})}{\Delta t}. \tag{3.35}$$

- **Euler backward:** this second-order accurate (in time) scheme is as follows:

$$\frac{\partial \phi_i(t)}{\partial t} = \frac{3\phi_i(t^n) - 4\phi_i(t^{n-1}) + \phi_i(t^{n-2})}{2\Delta t} \tag{3.36}$$



Draft
Milad Rakhsha

### 3.3.2 Advective Flux

The advective face flux in Eq. 3.34 may be written as follow [1]

$$\sum_{S_{ij}} (\boldsymbol{u}\phi) \cdot \boldsymbol{S_{ij}} = \sum_{S_{ij}} (\boldsymbol{u} \cdot \boldsymbol{S_{ij}}) \phi_{ij} = \sum_{S_{ij}} F_{ij}\ \phi_{ij}, \tag{3.37}$$

where $\phi_{ij}$ denotes the value of $\phi$ at the inter-face between cell $i$ and cell $j$. It can be calculated in various ways including [3, 4]:

- **Central Differencing:** This is a second-order accurate scheme and is a linear interpolation between $\phi_i$ and $\phi_j$, i.e.

$$\phi_{ij} = f_x \phi_i + (1 - f_x)\phi_j, \tag{3.38}$$

  where $f_x = d_i/d_{ij}$, and $d_{ij}$ is the distance between the center of cell $i$ and the center of cell $j$, and $d_i$ is the distance between the center of cell $i$ and the face between cell $i$ and cell $j$.

- **Upwind Differencing:** this is a first-order accurate scheme that acts as a numerical dissipation and increases stability by assuming that $\phi_{ij}$ is either $\phi_i$ or $\phi_j$ depending on the velocity direction at the face $ij$ as follows:

$$\phi_{ij} = \begin{cases} \phi_i & \text{if } F_{ij} = \boldsymbol{u} \cdot \boldsymbol{S_{ij}} \geq 0 \\ \phi_j & \text{if } F_{ij} = \boldsymbol{u} \cdot \boldsymbol{S_{ij}} < 0. \end{cases} \tag{3.39}$$

- **Blended Differencing:** This scheme is a combination of the central differencing and upwind differencing schemes and is as follows:

$$\phi_{ij} = \gamma \phi_{UD} + (1 - \gamma)\phi_{CD}, \tag{3.40}$$

  where $0 < \gamma < 1$ determines the contribution of the second-order method to the overall value.

### 3.3.3 Diffusive Flux

The diffusive face flux in Eq. 3.34 may be written as follow

$$\sum_{S_{ij}} (\boldsymbol{D}\nabla\phi)_{ij} \cdot \boldsymbol{S_{ij}} = \sum_{S_{ij}} (\boldsymbol{D}_{ij}\nabla\phi_{ij} \cdot \boldsymbol{S_{ij}}) = \sum_{S_{ij}} D_{ij} S_{ij} \frac{\phi_i - \phi_j}{d_{ij}}, \tag{3.41}$$

which is accurate if face normal is aligned with the line connecting cell $i$ and cell $j$, i.e. in orthogonal meshes. In case of non-orthogonal meshes additional corrections are required [3].

[1] The term $\boldsymbol{u} \cdot \boldsymbol{S_{ij}}$ in Eq. 3.37 is sometimes called *face flux*. This is denoted by *phi* in OpenFOAM.



Draft
Milad Rakhsha

### 3.3.4 Boundary Conditions

- Dirichlet type boundary conditions, i.e. $\phi\Big|_{S_D} = \phi_D$ (similarly a given advective flux $(\boldsymbol{u}\phi) \cdot S\Big|_{S_D} = f_D$) may be incorporated in Eq. 3.34 as follows:
  - $\phi_{ij} = \phi_D$ is substituted in Eq. 3.37 for any face on the Dirichlet part of the boundary.
  - For any face on the Dirichlet part of the boundary the, the gradient term associated with the diffusive flux in Eq. 3.41 is calculated as $S_{ij}\frac{\phi_i - \phi_j}{d_{ij}} = S_f\frac{\phi_i - \phi_D}{d}$, where $d$ is the distance between the boundary face and its cell center and $S_f$ is the magnitude area of the boundary face.
- Neumann type of boundary conditions, i.e. $\frac{\boldsymbol{S}}{S} \cdot \nabla\phi\Big|_{S_N} = g_\phi$ (similarly a given diffusive flux $(\boldsymbol{D}\nabla\phi) \cdot \boldsymbol{S}\Big|_{S_N} = f_D$) may be incorporated in Eq. 3.34 as follows:
  - For any external face on the Neumann part of the boundary, the face *values* of $\phi$ required in the advective flux term (see Eq. 3.37) can be interpolated from the face *gradient values* as $\phi_f = \phi_i + \boldsymbol{d} \cdot \nabla\phi_f = \phi_i + dg_\phi$.
  - For any face on the Neumann part of the boundary the, the gradient term associated with the diffusive flux in Eq. 3.41 is calculated as $\nabla\phi_f \cdot \boldsymbol{S_f} = g_\phi S_f$.



Draft
Milad Rakhsha

## 3.4 Finite Element Method

Where as the finite volume method (Section §3.3) in which *cell center* values are the degrees of freedom, in finite element method we choose the degrees of freedom to be defined at the *vertices* of the computational mesh. From this perspective, the finite element described herein is similar to the finite difference method. Subsequently, we seek out an approximate solution of $\phi$, i.e. $\bar{\phi}$ in the weak form of the conservation law stated in Eq. 3.2 such that :

$$\bar{\phi}(\boldsymbol{x}, t) = \sum_j \phi_j(t)\psi_j(\boldsymbol{x}), \tag{3.42}$$

where $\phi_j(t)$ denotes the nodal value of $\phi$ at node $j$, and $\psi_j(\boldsymbol{x})$ is a *shape* (also known as *interpolation* or *trail*) function associated with node $j$.

The following remarks clarify the rational behind this strategy:

- $\bar{\phi}(\boldsymbol{x}, t)$ indicates that $\bar{\phi}$ is a function of both space and time, while the nodal value $\phi_j(t)$ is only a function of time, and the basis function $\psi_j(\boldsymbol{x})$ is only a function of space. Hence, the separation of variables is done under the hood; the space functionality is taken into the shape functions, while the time functionality is taken into the nodal values.

- $\psi_j(\boldsymbol{x})$ is a special function; it is 1 if evaluated at the node $j$, and is 0 if evaluated at any other node $i$, i.e. $\psi_j(\boldsymbol{x}_j) = 1$ and $\psi_j(\boldsymbol{x}_i) = 0 \ \forall i \neq j$. Therefore, if we evaluate $\bar{\phi}(\boldsymbol{x}, t)$ at node $j$ it follows:

$$\begin{aligned}\bar{\phi}(\boldsymbol{x}_j, t) = \sum_j \phi_j(t)\psi_j(\boldsymbol{x}_i) =& \phi_1(t) \times \psi_1(\boldsymbol{x}_j) + \phi_2(t) \times \psi_2(\boldsymbol{x}_j) + \cdots + \phi_j(t) \times \psi_j(\boldsymbol{x}_j) + \cdots \\ =& \phi_1(t) \times 0 + \phi_2(t) \times 0 + \cdots + \phi_j(t) \times 1 + \cdots \qquad (3.43) \\ =& \phi_j(t), \qquad (3.44)\end{aligned}$$

  which is expected.

- $\psi_j(\boldsymbol{x})$ are special functions in that they are non-zero only in the cells surrounding the node/vertex $j$; elsewhere they are 0. Figure 9 illustrates this concept for linear interpolation functions.

- Eq. 3.42 states that the value of the $\phi$ can be obtained from a linear combination of basis functions $\psi_j(\boldsymbol{x})$; For instance, in a 1-dimensional problem such as the one shown in Figure 9, imagine we are interested to know the value of $\bar{\phi}(\boldsymbol{x}, t)$ at the the middle point between node $j$ and $j+1$, i.e. $x_m = \frac{x_j + x_{j+1}}{2}$. Note from Figure 9 that $\psi_i(x_m) \neq 0$ only for two shape functions; i.e. $\psi_j(x_m)$ and $\psi_{j+1}(x_m)$. Hence, the summation in Eq. 3.42 may be simplified to only two terms;

$$\begin{aligned}\bar{\phi}(x_m, t) =& \sum_j \phi_j(t)\psi_j(\boldsymbol{x}_m) \\ =& \phi_1(t) \times \psi_1(\boldsymbol{x}_m) + \phi_2(t) \times \psi_2(\boldsymbol{x}_m) + \cdots + \phi_j(t) \times \psi_j(\boldsymbol{x}_m) + \cdots \\ =& \phi_j(t) \times \psi_j(\boldsymbol{x}_m) + \phi_{j+1}(t) \times \psi_{j+1}(\boldsymbol{x}_m) \qquad (3.45)\end{aligned}$$



Draft
Milad Rakhsha

Because the $\psi_j(\boldsymbol{x})$ and $\psi_{j+1}(\boldsymbol{x})$ are both linear, we can see that the value of $\psi_j(\boldsymbol{x}_m) = 1/2$ and $\psi_{j+1}(\boldsymbol{x}_m) = 1/2$. Thus, $\bar{\phi}(x_m, t) = 1/2(\phi_j(t) + \phi_{j+1}(t))$ results from Eq. 3.45, which is in line with the expectation.

- Note that the use of linear interpolation functions should result in the same order of accuracy as we had in the finite difference method, due to linear approximation that was made in the discretization in Eqs. 3.11-3.14. The same holds for the finite volume method if upwind differencing is not used.

- Note that due to the separation of time function and space function in Eq. 3.42, all the space-related operation will be conducted on the $\psi_j(\boldsymbol{x})$s and all the time-related operations are performed on the $\phi_j(t)$s. For instance, if one needs to obtain the gradient or Laplacian of $\phi$ it follows that:

$$\nabla\bar{\phi}(\boldsymbol{x}, t) = \sum_j \phi_j(t)\nabla\psi_j(\boldsymbol{x}), \tag{3.46}$$

$$\nabla^2\bar{\phi}(\boldsymbol{x}, t) = \sum_j \phi_j(t)\nabla^2\psi_j(\boldsymbol{x}), \tag{3.47}$$

or for time-derivative of $\phi$ the following holds,

$$\frac{\partial\bar{\phi}(\boldsymbol{x}, t)}{\partial t} = \sum_j \frac{\partial\phi_j(t)}{\partial t}\psi_j(\boldsymbol{x}). \tag{3.48}$$

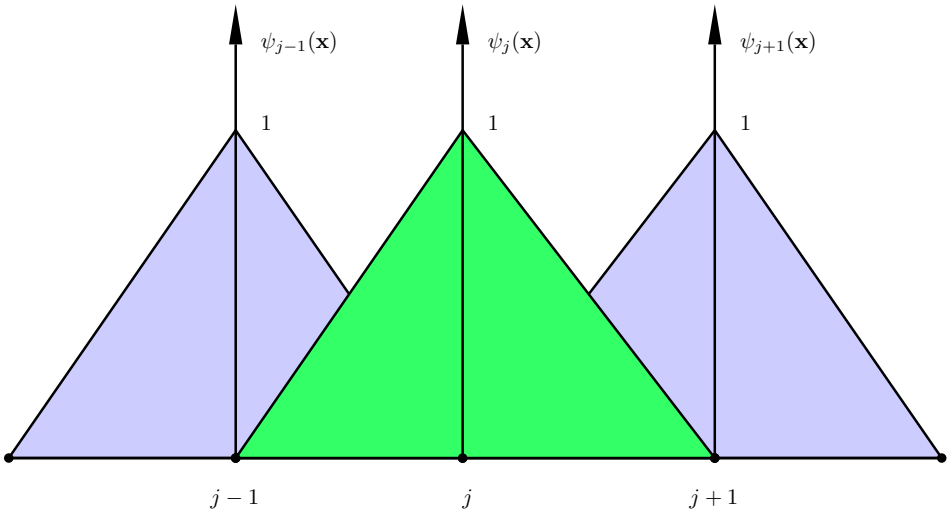



Figure 9: Representation of *piecewise linear* nodal basis (shape) functions in a 1D problem.



Draft
Milad Rakhsha

Now let's consider the weak form of the conservation law as mentioned in Eq. 3.2

$$\int_V w\Big(\frac{\partial \bar{\phi}}{\partial t} + \nabla \cdot (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) - s\Big)d\boldsymbol{x} = 0, \qquad \forall w \in \mathcal{W}.$$

After integration by parts and using the Divergence theorem we reached the following in Eq. 3.6 [1]:

$$\int_V (w\frac{\partial \bar{\phi}}{\partial t} - \nabla w \cdot (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) - ws)d\boldsymbol{x} + \int_{S_N} w(\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) \cdot \boldsymbol{n}\, ds = 0.$$

Remember that in the finite volume method the conservation laws are solved for each cell (control volume) in the computational domain. In finite element we solve one equation per vertex. Now assume that we would like to solve the weak form of the conservation law on vertex $i$.

### 3.4.1 Choice of the Weight Function

Many functions may be chosen as the test/weight function. As explained in section §3.1, the choice of $w = \delta(\boldsymbol{x} - \boldsymbol{x}_0)$ leads to satisfying the strong form of the equation (Eq. 3.9), which is the idea of finite difference method. This is also known as *collocation* technique.

In the *subdomamin* method, chooses $w = 1$ in each element and $w = 0$ outside of each element. This consequently leads to the integral form of the equations (Eq.3.8), which is used in finite volume method.

In the least square method, the weight function is chosen to be the derivative of the residual equation w.r.t to an unknown coefficient c. Specifically, in Eq. 3.1, $w = \partial R/\partial c$ is chosen. Consequently, Eq. 3.2 leads to:

$$\begin{aligned} &\int_V w\mathcal{R}(\bar{\phi})d\boldsymbol{x} = 0, \qquad \forall w \in \mathcal{W} \\ &\int_V \frac{\partial R}{\partial c}\mathcal{R}d\boldsymbol{x} = \frac{1}{2}\frac{\partial}{\partial c}\int_V \mathcal{R}^2 d\boldsymbol{x} = 0, \end{aligned} \tag{3.49a}$$

which implies that *average squared residual* is to be minimized with respect to fitting coefficients $c$.

Finally, in the Galerkin method, the weight function is chosen to be the for vertex $i$ is the same as the interpolation/trail function, i.e. $w = \psi_i$. This is the method often used for developing finite element equations for field problems and shall be discussed in this note.

### 3.4.2 Galerkin Method

In the *Galerkin* method, the weight/test function for vertex $i$ is the same as the interpolation function, i.e. $w = \psi_i$. Using Eqs. 3.46, 3.47, and 3.48 and substituting $w = \psi_i$ and $\nabla w = \nabla\psi_i$ into Eq. 3.6 the following is obtained :



Draft
Milad Rakhsha

$$\sum_j \int_V \Big(\psi_i(\boldsymbol{x})\frac{\partial \phi_j(t)}{\partial t}\psi_j(\boldsymbol{x}) - \nabla\psi_i(\boldsymbol{x})\cdot(\boldsymbol{u}_j\phi_j(t)\psi_j(\boldsymbol{x}) - \boldsymbol{D}\phi_j(t)\nabla\psi_j(\boldsymbol{x}))\Big)d\boldsymbol{x}$$
$$-\int_V \psi_i(\boldsymbol{x})s(\boldsymbol{x})d\boldsymbol{x} + \int_{S_N}\psi_i(\boldsymbol{x})(\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds = 0,$$
$$\sum_j \Big(\int_V \psi_i\psi_j d\boldsymbol{x}\Big)\frac{\partial\phi_j(t)}{\partial t} - \sum_j\Big(\int_V \nabla\psi_i\cdot(\boldsymbol{u}_j\psi_j - \boldsymbol{D}\nabla\psi_j)d\boldsymbol{x}\Big)\phi_j(t)$$
$$-\int_V \psi_i s d\boldsymbol{x} + \int_{S_N}\psi_i(\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds = 0 \tag{3.50}$$

which the case if the total flux $\boldsymbol{f} = (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}$ is known on the Neumann part of the boundary. If only the diffusive flux ($\boldsymbol{f} = (-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}$) is prescribed on the Neumann part of the boundary, then the integration by parts is done only on that diffusive flux in Eq. 3.2 which leads to:

$$\int_V (w\frac{\partial\bar{\phi}}{\partial t} + w\nabla\cdot(\boldsymbol{u}\bar{\phi}) + \nabla w\cdot(\boldsymbol{D}\nabla\bar{\phi}) - ws)d\boldsymbol{x} + \int_{S_N} w(-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds = 0. \tag{3.51}$$

The divergence of the advective term is approximated as :

$$\nabla\cdot(\boldsymbol{u}\bar{\phi}) = \sum_j \phi_j(t)(\boldsymbol{u}_j\cdot\nabla\psi_j(\boldsymbol{x})), \tag{3.52}$$

Using Eqs. 3.46, 3.48, 3.52 and substituting $w = \psi_i$ and $\nabla w = \nabla\psi_i$ into Eq. 3.51, we get the following algebraic equation for the conservation law on the node $i$:

$$\sum_j \Big(\int_V \psi_i\psi_j d\boldsymbol{x}\Big)\frac{\partial\phi_j(t)}{\partial t} + \sum_j\Big(\int_V \psi_i(\boldsymbol{u}_j\cdot\nabla\psi_j) + \nabla\psi_i\cdot(\boldsymbol{D}\nabla\psi_j)d\boldsymbol{x}\Big)\phi_j(t)$$
$$-\int_V \psi_i s d\boldsymbol{x} + \int_{S_N}\psi_i(-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds = 0 \tag{3.53}$$

Defining the following terms,

$$m_{ij} = \int_V \psi_i\psi_j d\boldsymbol{x} \tag{3.54}$$
$$\boldsymbol{c}_{ij} = \int_V \psi_i\nabla\psi_j d\boldsymbol{x} \tag{3.55}$$
$$d_{ij} = \int_V \nabla\psi_i\cdot(\boldsymbol{D}\nabla\psi_j)d\boldsymbol{x} \tag{3.56}$$
$$r_i^{full} = \int_V \psi_i s d\boldsymbol{x} - \int_{S_N}\psi_i(\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds \tag{3.57}$$
$$r_i^{diff} = \int_V \psi_i s d\boldsymbol{x} - \int_{S_N}\psi_i(-\boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}\,ds \tag{3.58}$$



Draft
Milad Rakhsha

We can simplify Eq. 3.50 using Eqs. 3.54-3.56 and Eq. 3.57 to get the final discrete form of the conservation law on node $i$ when the total flux is prescribed on the Neumann part of the boundary

$$\sum_j m_{ij}\frac{\partial \phi_j(t)}{\partial t} + \sum_j \Big( - \boldsymbol{u}_j \cdot \boldsymbol{c}_{ij} + d_{ij}\Big)\phi_j(t) - r_i^{full} = 0. \tag{3.59}$$

We can simplify Eq. 3.53 using Eqs. 3.54-3.56 and Eq. 3.58 to get the final discrete form of the conservation law on node $i$ when the only the diffusive flux is prescribed on the Neumann part of the boundary

$$\sum_j m_{ij}\frac{\partial \phi_j(t)}{\partial t} + \sum_j \Big(\boldsymbol{u}_j \cdot \boldsymbol{c}_{ij} + d_{ij}\Big)\phi_j(t) - r_i^{diff} = 0. \tag{3.60}$$

If we put together these equations for all of the vertices in the computational mesh, we can write the matrix-form of Eqs. 3.59 and 3.60 as follows:

$$\boldsymbol{M}\frac{\partial \phi(t)}{\partial t} - \boldsymbol{K}\phi(t) - \boldsymbol{r} = 0, \tag{3.61}$$

where $\boldsymbol{M} = \{m_{ij}\}$, $\boldsymbol{K} = \{k_{ij}\}$. Above, $\boldsymbol{r} = \{r_i^{full}\}$, $k_{ij} = \boldsymbol{u}_j \cdot \boldsymbol{c}_{ij} - d_{ij}$ in Eq. 3.59 whereas $\boldsymbol{r} = \{r_i^{diff}\}$ and $k_{ij} = -(\boldsymbol{u}_j \cdot \boldsymbol{c}_{ij} + d_{ij})$ for Eq. 3.60.

### 3.4.3 Enforcement of Dirichlet Condition

There are two ways of enforcing the Dirichlet boundary conditions:

- **Strong Enforcement** : The algebraic equation 3.60, or 3.59 is replaced with $\phi_i = \phi_D(\boldsymbol{x}_i)$ on any node that is on the Dirichlet part of the boundary ($S_D$).
- **Weak Enforcement** : In the algebraic equation 3.59, specifically in the $r_i^{full}$ term, the $(\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi})\cdot\boldsymbol{n}$ is set to be $(\boldsymbol{u}\phi_D(\boldsymbol{x}_i))\cdot\boldsymbol{n}$ on any node $i$ that is on the Dirichlet part of the boundary ($S_D$), i.e. the Dirichlet boundary condition is treated as a Neumann condition. Note that this does not enforce $\phi_i = \phi_D(\boldsymbol{x}_i)$ strongly, but in a weak sense as this is satisfied only in an integral sense over the boundary.

### 3.4.4 Calculation of Element and System Matrices

We turn our attention to the calculation of integrals in Eqs. 3.54-3.58. Note that these integral are to be calculated on the whole domain, i.e. $V$. However, due to compact/limited support radius of the shape functions (see Figure 9) associated to each node $i$, calculation of these integrals for node $i$ is only necessary over the cells/elements that are connected to node $i$; elsewhere the integrals are zero because $\psi_i$ is zero (see Figure 10). This is the case when calculating load vectors such as $r_i^{full}$ and $r_i^{diff}$. Similarly, the effective domain of non-zero



Draft
Milad Rakhsha

contribution to $m_{ij}$, $\boldsymbol{c}_{ij}$, and $d_{ij}$ is the region where both $\psi_i$ and $\psi_j$ are non-zero, i.e. the elements that share the segment $ij$ as shown in Figure 10.

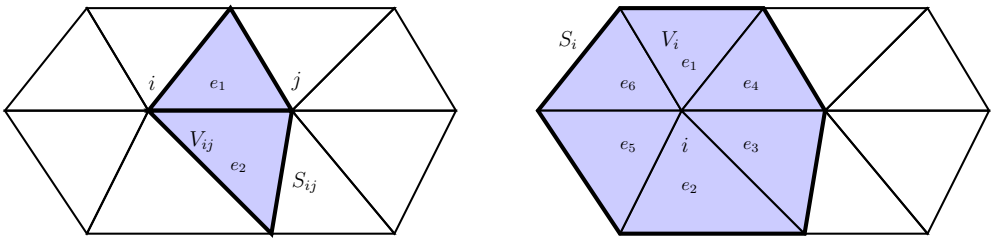



Figure 10: The domain of nonzero value in $m_{ij}$, $\boldsymbol{c}_{ij}$, and $d_{ij}$ (left). The domain of calculation associated with node $i$ (right)

For instance, we can calculate the $m_{ij}$ coefficient associated with the mesh illustrated Figure 10 as follows:

$$m_{ij} = \int_V \psi_i \psi_j d\boldsymbol{x} = \int_{e_1} \psi_i \psi_j d\boldsymbol{x} + \int_{e_2} \psi_i \psi_j d\boldsymbol{x}. \tag{3.62}$$

Similarly, the load $r_i^{full}$ may be calculated as

$$\begin{aligned} r_i = &\int_V \psi_i s d\boldsymbol{x} - \int_{S_N} \psi_i (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) \cdot \boldsymbol{n}\, ds \\ &\int_{e_1} \psi_i s V + \int_{e_2} \psi_i s V + \cdots + \int_{e_6} \psi_i s V + - \int_{S_N \cap S_i} \psi_i (\boldsymbol{u}\bar{\phi} - \boldsymbol{D}\nabla\bar{\phi}) \cdot \boldsymbol{n}\, ds \end{aligned} \tag{3.63}$$

Thus, we conclude that the $m_{ij}$, $\boldsymbol{c}_{ij}$, $d_{ij}$, and $r_i$ coefficients require element-wise calculations of the integrals.

In the following we describe the process of building the element-wise matrices and assembly of the global matrices. For each element $e_k$, build the mass $\boldsymbol{M}_{e_k}$, advection $\boldsymbol{C}_{e_k}$ and diffusion $\boldsymbol{D}_{e_k}$ matrices and load vector $\boldsymbol{r}_{e_k}$ as follows:



Draft
Milad Rakhsha

$$\boldsymbol{M}_{e_k} = \int_{e_k} \begin{bmatrix} \psi_1\psi_1 & \psi_1\psi_2 & \cdots \\ \psi_2\psi_1 & \psi_2\psi_2 & \cdots \\ \psi_3\psi_1 & \psi_3\psi_2 & \cdots \\ \vdots & \vdots & \vdots \\ \psi_n\psi_1 & \psi_n\psi_2 & \cdots \end{bmatrix} d\boldsymbol{x} \tag{3.64}$$

$$\boldsymbol{C}_{e_k} = \int_{e_k} \begin{bmatrix} \psi_1\nabla\psi_1 & \psi_2\nabla\psi_2 & \cdots \\ \psi_2\nabla\psi_1 & \psi_2\nabla\psi_2 & \cdots \\ \psi_3\nabla\psi_1 & \psi_2\nabla\psi_2 & \cdots \\ \vdots & \vdots & \vdots \\ \psi_n\nabla\psi_1 & \psi_n\nabla\psi_2 & \cdots \end{bmatrix} d\boldsymbol{x} \tag{3.65}$$

$$\boldsymbol{D}_{e_k} = \int_{e_k} \begin{bmatrix} \nabla\psi_1 \cdot (\boldsymbol{D}\nabla\psi_1) & \nabla\psi_2 \cdot (\boldsymbol{D}\nabla\psi_2) & \cdots \\ \nabla\psi_2 \cdot (\boldsymbol{D}\nabla\psi_1) & \nabla\psi_2 \cdot (\boldsymbol{D}\nabla\psi_2) & \cdots \\ \nabla\psi_3 \cdot (\boldsymbol{D}\nabla\psi_1) & \nabla\psi_2 \cdot (\boldsymbol{D}\nabla\psi_2) & \cdots \\ \vdots & \vdots & \vdots \\ \nabla\psi_n \cdot (\boldsymbol{D}\nabla\psi_1) & \nabla\psi_n \cdot (\boldsymbol{D}\nabla\psi_2) & \cdots \end{bmatrix} d\boldsymbol{x} \tag{3.66}$$

$$\boldsymbol{r}_{e_k}^{full} = \int_{e_k} \begin{bmatrix} \psi_1 s \\ \psi_2 s \\ \psi_3 s \\ \vdots \\ \psi_n s \end{bmatrix} d\boldsymbol{x} - \int_{S_N \cap S_{e_k}} \begin{bmatrix} \psi_1(\boldsymbol{u}_1\phi_1 - \boldsymbol{D}\nabla\phi_1)\cdot\boldsymbol{n} \\ \psi_2(\boldsymbol{u}_2\phi_2 - \boldsymbol{D}\nabla\phi_2)\cdot\boldsymbol{n} \\ \psi_3(\boldsymbol{u}_2\phi_3 - \boldsymbol{D}\nabla\phi_3)\cdot\boldsymbol{n} \\ \vdots \\ \psi_n(\boldsymbol{u}_n\phi_n - \boldsymbol{D}\nabla\phi_n)\cdot\boldsymbol{n} \end{bmatrix} ds \tag{3.67}$$

$$\boldsymbol{r}_{e_k}^{diff} = \int_{e_k} \begin{bmatrix} \psi_1 s \\ \psi_2 s \\ \psi_3 s \\ \vdots \\ \psi_n s \end{bmatrix} d\boldsymbol{x} - \int_{S_N \cap S_{e_k}} \begin{bmatrix} \psi_1(-\boldsymbol{D}\nabla\phi_1)\cdot\boldsymbol{n} \\ \psi_2(-\boldsymbol{D}\nabla\phi_2)\cdot\boldsymbol{n} \\ \psi_3(-\boldsymbol{D}\nabla\phi_3)\cdot\boldsymbol{n} \\ \vdots \\ \psi_n(-\boldsymbol{D}\nabla\phi_n)\cdot\boldsymbol{n} \end{bmatrix} ds \tag{3.68}$$

The size of the integrals depends on the shape of the elements and the number of degrees of freedom in the element. For instance, for triangular elements the size of matrices are $3\times3$ for linear shape functions. Once these local (element) matrices/vector are computed, they serve as the partial contribution of the element to the global matrices/vector as follows. Consider that the $G_i$ and $G_j$ are the global nodes corresponding to the local nodes $i$ and $j$ in the element $e_k$. Hence, $\boldsymbol{M}_{e_k}(i,j)$, $\boldsymbol{C}_{e_k}(i,j)$ and $\boldsymbol{D}_{e_k}(i,j)$ are the contribution of the element $e_k$ to the global element $\boldsymbol{M}(G_i,G_j)$, $\boldsymbol{C}(G_i,G_j)$, and $\boldsymbol{D}(G_i,G_j)$ matrices, respectively. Similarly, we could make the same statement for the load vector $\boldsymbol{r}_{e_k}$; $\boldsymbol{r}_{e_k}(i)$ is the contribution of the element $e_k$ to the global element $\boldsymbol{r}(G_i)$. Note, also that the elements of the second integral in Eq. **??** are non-zero if a local node to element $e_k$ is on the Neumann part of the boundary.

Note also that each element of $\boldsymbol{C}_{e_k}(i,j)$ is a triplet (in 3D) due to the $\nabla$ operator. These elements later are going to be dot produced (see Eq. 3.60 or 3.68) with the velocity to create



Draft
Milad Rakhsha

scalar values.

### 3.4.5 Numerical Integration of the Elements' Matrices/Vectors

We turn our attention to the numerical calculation of the element matrices in Eqs. 3.64-3.68. These integrals needs to be calculated for every element in the domain, regardless of their shape. In order to make these calculations numerically efficient we make an assumption about the elements. This assumption allows us transform the integrals in Eqs. 3.64-3.68 from a *physical space* to a *natural space*, and to calculate the integrals in a domain which is the same for all elements, regardless of their physical shape. Herein, we assume the elements are *isoparametric* elements, i.e. elements where the shape and element field quantities are defined by the same interpolation functions. Remember that we used Eq. 3.42 for interpolation of the field value $\phi(\boldsymbol{x}, t)$ as

$$\phi(\boldsymbol{x}, t) = \sum_j \phi_j(t)\psi_j(\boldsymbol{x}), \tag{3.69}$$

where $\psi_j(\boldsymbol{x})$ are the shape/basis functions associated with nodes $j$. In the isoparametric elements we use the same interpolation functions to interpolate the coordinates of the elements, but in terms of natural coordinates $\boldsymbol{\zeta} = (\zeta, \eta, \chi)$

$$\boldsymbol{x}(\boldsymbol{\zeta}, t) = \sum_j \boldsymbol{x}_j(t)\psi_j(\boldsymbol{\zeta}). \tag{3.70}$$

Let's assume a Quadrilateral element in physical and natural spaces as shown in Figure 11.

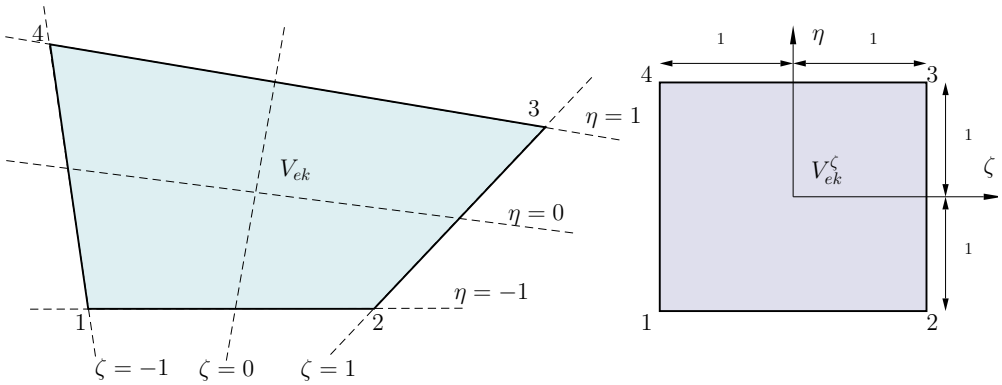



Figure 11: left : A four-node plane element in physical space ($\boldsymbol{x}$ domain). right : the same element in the natural space ($\boldsymbol{\zeta}$ domain).

The interpolation function in the natural coordinate system for bilinear Quadrilateral



Draft
Milad Rakhsha

element is defined via:

$$\psi_1(\boldsymbol{\zeta}) = \psi_1(\zeta, \eta) = \frac{1}{4}(1-\zeta)(1-\eta) \tag{3.71}$$

$$\psi_2(\boldsymbol{\zeta}) = \psi_2(\zeta, \eta) = \frac{1}{4}(1+\zeta)(1-\eta) \tag{3.72}$$

$$\psi_3(\boldsymbol{\zeta}) = \psi_3(\zeta, \eta) = \frac{1}{4}(1+\zeta)(1+\eta) \tag{3.73}$$

$$\psi_4(\boldsymbol{\zeta}) = \psi_4(\zeta, \eta) = \frac{1}{4}(1-\zeta)(1+\eta) \tag{3.74}$$

Note that these interpolation function satisfy the properties mentioned in Section §3.4, e.g,

- the shape functions are 1 at their corresponding vertex. For instance, $\psi_4(\boldsymbol{\zeta}_4) = \psi_4(-1,+1) = 1$, $\psi_3(\boldsymbol{\zeta}_3) = \psi_3(+1,+1) = 1$, $\psi_2(\boldsymbol{\zeta}_2) = \psi_2(+1,-1) = 1$, and $\psi_1(\boldsymbol{\zeta}_1) = \psi_1(-1,-1) = 1$.
- the shape functions of any node is 0 on any node other than their corresponding node. For instance, $\psi_4(\boldsymbol{\zeta}_3) = \psi_4(+1,+1) = 0$, $\psi_4(\boldsymbol{\zeta}_2) = \psi_4(+1,-1) = 0$, and $\psi_4(\boldsymbol{\zeta}_1) = \psi_4(-1,-1) = 0$ as shown in Figure 11.

Subsequently, we use the following identity to transform the integral in Eqs. 3.64-3.68 to natural coordinates,

$$\int f(u)du = \int f(g(v))\frac{dg}{dv}dv.$$

Substituting $u = \boldsymbol{x}$, $v = \boldsymbol{\zeta}$, and $g(v) = \boldsymbol{x}(\boldsymbol{\zeta})$, we obtain:

$$\int_{V_e} f(\boldsymbol{x})d\boldsymbol{x} = \int_{V_e^n} f(\boldsymbol{x}(\boldsymbol{\zeta}))|\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}|d\boldsymbol{\zeta}. \tag{3.75}$$

Above, $\boldsymbol{J} = \frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}} = \frac{\partial \boldsymbol{x}}{\partial \boldsymbol{\zeta}} = \frac{\partial(x,y,z)}{\partial(\zeta,\eta,\chi)}$ denotes the transformation Jacobian matrix and $|\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}|$ represent the determinant of the Jacobian matrix. It arises because of the domain transformation and is closely related to the chain rule of partial differentiation as follows:

$$\frac{\partial}{\partial\zeta} = \frac{\partial}{\partial x}\frac{\partial x}{\partial\zeta} + \frac{\partial}{\partial y}\frac{\partial y}{\partial\zeta} + \frac{\partial}{\partial z}\frac{\partial z}{\partial\zeta}, \tag{3.76}$$

$$\frac{\partial}{\partial\eta} = \frac{\partial}{\partial x}\frac{\partial x}{\partial\eta} + \frac{\partial}{\partial y}\frac{\partial y}{\partial\eta} + \frac{\partial}{\partial z}\frac{\partial z}{\partial\eta}, \tag{3.77}$$

$$\frac{\partial}{\partial\chi} = \frac{\partial}{\partial x}\frac{\partial x}{\partial\chi} + \frac{\partial}{\partial y}\frac{\partial y}{\partial\chi} + \frac{\partial}{\partial z}\frac{\partial z}{\partial\chi}, \tag{3.78}$$



Draft
Milad Rakhsha

or in the matrix form:

$$\frac{d}{d\boldsymbol{\zeta}} = \frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}} \frac{d}{d\boldsymbol{x}}, \tag{3.79}$$

$$\begin{bmatrix} \frac{\partial}{\partial \zeta} \\ \frac{\partial}{\partial \eta} \\ \frac{\partial}{\partial \chi} \end{bmatrix} = \begin{bmatrix} \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} & \frac{\partial z}{\partial \zeta} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \chi} & \frac{\partial y}{\partial \chi} & \frac{\partial z}{\partial \chi} \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix} = \boldsymbol{J} \begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix}. \tag{3.80}$$

Above, $\boldsymbol{J}$ is the transformation Jacobian matrix in 3D. We can make the same argument to show that the Jacobian matrix in 2D is

$$\boldsymbol{J} = \begin{bmatrix} \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} \end{bmatrix}, \tag{3.81}$$

and is

$$\boldsymbol{J} = \begin{bmatrix} \frac{\partial x}{\partial \zeta} \end{bmatrix}, \tag{3.82}$$

in 1D. Conversely, Eq. 3.79 may be written for $\frac{d}{d\boldsymbol{x}}$ as follows:

$$\begin{aligned} \frac{d}{d\boldsymbol{x}} &= (\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}})^{-1} \frac{d}{d\boldsymbol{\zeta}}, \\ \frac{d}{d\boldsymbol{x}} &= \boldsymbol{J}^{-1} \frac{d}{d\boldsymbol{\zeta}}, \end{aligned} \tag{3.83}$$

which is used to transform the space operators such as gradient/divergence from the physical space to the natural space. For instance, the Gradient operator $\nabla$ may be transformed from the physical domain $(\frac{d}{d\boldsymbol{x}})$ into the natural domain $(\frac{d}{d\boldsymbol{\zeta}})$ via :

$$\nabla = \begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix} = \frac{d}{d\boldsymbol{x}} = \boldsymbol{J}^{-1} \frac{d}{d\boldsymbol{\zeta}} = \boldsymbol{J}^{-1} \nabla_{\zeta} = \boldsymbol{J}^{-1} \begin{bmatrix} \frac{\partial}{\partial \zeta} \\ \frac{\partial}{\partial \eta} \\ \frac{\partial}{\partial \chi} \end{bmatrix} \tag{3.84}$$

Note that the Jacobian of the matrix and also its inverse come into play in calculation of the element integrals as will be explained shortly. Thus, we need to explain how the Jacobian matrix is computed. We explain the procedure in 3D but it is applicable to 2D and 1D in a similar fashion. The Jacobian matrix may be computed using Eq. 3.70 and considering that



Draft
Milad Rakhsha

$\boldsymbol{x} = (x, y, z)$ and $\boldsymbol{\zeta} = (\zeta, \eta, \chi)$ as follows

$$\boldsymbol{J} = \boldsymbol{J}(\boldsymbol{\zeta}) = \boldsymbol{J}(\zeta, \eta, \chi) = \begin{bmatrix} \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} & \frac{\partial z}{\partial \zeta} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \chi} & \frac{\partial y}{\partial \chi} & \frac{\partial z}{\partial \chi} \end{bmatrix}$$
$$= \begin{bmatrix} \sum_j x_j(t) \frac{\partial \psi_j}{\partial \zeta} & \sum_j y_j(t) \frac{\partial \psi_j}{\partial \zeta} & \sum_j z_j(t) \frac{\partial \psi_j}{\partial \zeta} \\ \sum_j x_j(t) \frac{\partial \psi_j}{\partial \eta} & \sum_j y_j(t) \frac{\partial \psi_j}{\partial \eta} & \sum_j z_j(t) \frac{\partial \psi_j}{\partial \eta} \\ \sum_j x_j(t) \frac{\partial \psi_j}{\partial \chi} & \sum_j y_j(t) \frac{\partial \psi_j}{\partial \chi} & \sum_j z_j(t) \frac{\partial \psi_j}{\partial \chi} \end{bmatrix} \tag{3.85}$$

Note that if shape functions are linear w.r.t. $\boldsymbol{\zeta} = (\zeta, \eta, \chi)$, then the above integral is constant no matter at what location in the elements it is computed. This is the case for elements with linear shape functions such as linear beams, bilinear Quadrilateral, and trilinear Hexahedron elements. This is important because as will be explained shortly, this Jacobian matrix should be evaluated at certain points (Guass-Quadrature) inside the element for numerical approximation of the integrals. Having a constant Jacobian matrix reduces the number of Jacobian evaluation per element.

Having explained the transformation from the physical coordinates to natural coordinates, we can transform the element matrices from the physical coordinates to natural coordinates using 3.75. For instance, consider the integration of the first element of the mass matrix associated with the local node 1 in Eq. 3.64, i.e.,

$$\begin{aligned} \int_{V_{e_k}} \psi_1(\boldsymbol{x}) \psi_1(\boldsymbol{x}) d\boldsymbol{x} &= \int_{V^{\boldsymbol{\zeta}}_{e_k}} \psi_1(\boldsymbol{x}(\boldsymbol{\zeta})) \psi_2(\boldsymbol{x}(\boldsymbol{\zeta})) |\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}| d\boldsymbol{\zeta} \\ &= \int_{\eta=-1}^{\eta=1} \int_{\zeta=-1}^{\zeta=1} \psi_1(\boldsymbol{\zeta}) \psi_2(\boldsymbol{\zeta}) |\boldsymbol{J}| \, d\zeta d\eta. \end{aligned} \tag{3.86}$$

Similarly, the first element of the Advection matrix associated with the local node 1 in element $e_k$ in Eq. 3.65 may be computed as

$$\begin{aligned} \int_{V_{e_k}} \psi_1(\boldsymbol{x}) \nabla \psi_1(\boldsymbol{x}) d\boldsymbol{x} &= \int_{V^{\boldsymbol{\zeta}}_{e_k}} \psi_1(\boldsymbol{x}(\boldsymbol{\zeta})) \Big( \frac{d\boldsymbol{\zeta}}{d\boldsymbol{x}} \nabla_{\boldsymbol{\zeta}} \psi_1\big(\boldsymbol{x}(\boldsymbol{\zeta})\big) \Big) |\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}| d\boldsymbol{\zeta} \\ &= \int_{\eta=-1}^{\eta=1} \int_{\zeta=-1}^{\zeta=1} \psi_1(\boldsymbol{\zeta}) \Big( \boldsymbol{J}^{-1} \nabla_{\boldsymbol{\zeta}} \psi_1(\boldsymbol{\zeta}) \Big) |\boldsymbol{J}| \, d\zeta d\eta. \end{aligned} \tag{3.87}$$

The first element of the Diffusion matrix associated with the local node 1 in element $e_k$ in



Draft
Milad Rakhsha

Eq. 3.66 may be computed similarly as

$$\int_{V_{e_k}} \nabla\psi_1(\boldsymbol{x}) \cdot (\boldsymbol{D}\nabla\psi_1(\boldsymbol{x}))d\boldsymbol{x} = \int_{V_{e_k}^{\boldsymbol{\zeta}}} \left( \left(\frac{d\boldsymbol{\zeta}}{d\boldsymbol{x}}\nabla_{\boldsymbol{\zeta}}\psi_1\big(\boldsymbol{x}(\boldsymbol{\zeta})\big)\right) \cdot \boldsymbol{D}\left(\frac{d\boldsymbol{\zeta}}{d\boldsymbol{x}}\nabla_{\boldsymbol{\zeta}}\psi_1\big(\boldsymbol{x}(\boldsymbol{\zeta})\big)\right)\right) |\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}| d\boldsymbol{\zeta}$$
$$= \int_{\eta=-1}^{\eta=1}\int_{\zeta=-1}^{\zeta=1} \left( \left(\boldsymbol{J}^{-1}\nabla_{\boldsymbol{\zeta}}\psi_1(\boldsymbol{\zeta})\right) \cdot \boldsymbol{D}\left(\boldsymbol{J}^{-1}\nabla_{\boldsymbol{\zeta}}\psi_1(\boldsymbol{\zeta})\right)\right) |\boldsymbol{J}|\, d\zeta d\eta, \tag{3.88}$$

Elements of load vector in Eqs. 3.67-3.68 are calculated as follows,

$$\int_{V_{e_k}} \psi_1(\boldsymbol{x}) s(\boldsymbol{x}) d\boldsymbol{x} = \int_{V_{e_k}^{\boldsymbol{\zeta}}} \psi_1\big(\boldsymbol{x}(\boldsymbol{\zeta})\big) s\big(\boldsymbol{x}(\boldsymbol{\zeta})\big) |\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}| d\boldsymbol{\zeta} = \int_{\eta=-1}^{\eta=1}\int_{\zeta=-1}^{\zeta=1} \psi_1(\boldsymbol{\zeta}) s(\boldsymbol{\zeta}) |\boldsymbol{J}|\, d\zeta d\eta. \tag{3.89}$$

Similarly, the surface integral (line integral in 2D) that comes into play if the element has a segment on the boundary

$$\int_{S_N \cap S_{e_k}} \psi_1(\boldsymbol{x}) h d\boldsymbol{x} = \int_{S_N \cap S_{e_k}^{\boldsymbol{\zeta}}} \psi_1\big(\boldsymbol{x}(\boldsymbol{\zeta})\big) h |\frac{d\boldsymbol{x}}{d\boldsymbol{\zeta}}| d\boldsymbol{\zeta} = \int_{\zeta=-1}^{\zeta=1} \psi_1(\boldsymbol{\zeta}) h |\boldsymbol{J}|\, d\zeta, \tag{3.90}$$

where $h = (\boldsymbol{u}\phi - \boldsymbol{D}\nabla\phi_1)\cdot\boldsymbol{n}$ or $h = (-\boldsymbol{D}\nabla\phi_1)\cdot\boldsymbol{n}$ is the boundary flux on the Neumann part of the boundary. Above, Eq. 3.84 was used to transform the Gradient operator $\nabla$ from the physical domain $(\frac{d}{d\boldsymbol{x}})$ into the natural domain $(\frac{d}{d\boldsymbol{\zeta}})$ via $\nabla = \boldsymbol{J}^{-1}\nabla_{\boldsymbol{\zeta}}$.

Having transformed the element matrices from the physical coordinates to natural coordinates, we proceed with numerically integrating Eqs. 3.86-3.88 via Gauss-Quadrature (GQ) method. According to Gauss-Quadrature method,

$$\int_{x=-1}^{x=+1} f(x)dx = \sum_{i=1}^{n} w_i f(x_i), \tag{3.91}$$

i.e., the integral of the function may be converted to the summation of weighted values of the function evaluated at the gauss-quadrature points. The numerical values of the gauss points and weights are shown in Table 1. Note that

- higher order approximation require more gauss-quadrature points,
- $n$ quadrature points can exactly compute the integral for a polynomial function $f$ of degree $2n-1$ or less, e.g., 3 points approximation can exactly compute the integral of a polynomial of degree 5.



Draft
Milad Rakhsha

| n | $x_i$ | $w_i$ |
|---|---|---|
| 1 | 0 | 2 |
| 2 | $-\frac{1}{\sqrt{3}}$, $+\frac{1}{\sqrt{3}}$ | -0.57735,-0.57735 |
| 3 | $-\frac{3}{\sqrt{5}}$, 0, $+\frac{3}{\sqrt{5}}$ | $\frac{5}{9}$,$\frac{8}{9}$,$\frac{5}{9}$ |

Table 1: Gauss-Quadrature points and weights up to 3-point approximation

We can generalize the GQ method to higher dimensional integrals as follows:

$$\int_{y=-1}^{y=+1}\int_{x=-1}^{x=+1} f(x,y)dxdy = \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j f(x_i, y_j), \tag{3.92}$$

$$\int_{z=-1}^{z=+1}\int_{y=-1}^{y=+1}\int_{x=-1}^{x=+1} f(x,y)dxdydz = \sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} w_i w_j w_k f(x_i, y_j, z_k). \tag{3.93}$$

Ultimately, the calculation of the elements' matrices come down to evaluation of the integrands in Eqs. 3.86-3.88 at Guass-Quadrature points inside the elements.



Draft
Milad Rakhsha

## 3.5 Smoothed Particle Hydrodynamics

In contrast to Eulerian discretization methods discussed in Sections §3.2-§3.4 where the degrees of freedom (DOF) are stationary observers, Smoothed Particle Hydrodynamics (SPH) is a Lagrangian method. Hence, in SPH the DOF are moving observers.

In order to approximate a function $f$ at location $\mathbf{r}_i$ as [5]

$$f(\mathbf{r}_i) \approx \langle f \rangle_i = \sum_{j \in \mathcal{S}(i)} \frac{m_j}{\rho_j} f(\mathbf{r}_j) W_{ij} \ , \tag{3.94}$$

where $\langle f \rangle_i$ indicates the SPH approximation of $f$ at the location of SPH particle, or marker, $i$; $\mathcal{S}(i)$ represents the collection of SPH particles found in the support domain associated with particle $i$; $\rho_j$ is the density $\rho(\mathbf{r}_j)$ at location $\mathbf{r}_j$ of particle $j$; $m_j = \rho_j V_j$ and $V_j = (\sum_{k \in \mathcal{S}(j)} W_{jk})^{-1}$ are the mass and volume associated with marker $j$, respectively; $W_{ij} \equiv W(|\boldsymbol{r}_i - \mathbf{r}_j|, h)$, where $|\mathbf{r}|$ is the length of $\mathbf{r}$. The kernel function $W$ can assume various expressions, e.g., a cubic spline kernel for 3D problems:

$$W(|\mathbf{r}|, h) = \frac{5}{14\pi h^3} \times \begin{cases} (2-q)^3 - 4(1-q)^3, & 0 \le q < 1 \\ (2-q)^3, & 1 \le q < 2 \\ 0, & q \ge 2 \end{cases} , \tag{3.95}$$

where, if the kernel function is located at the origin, $q \equiv |\mathbf{r}|/h$. The radius of the support domain, $\kappa h$, is proportional to the characteristic length $h$ through the parameter $\kappa$, the latter commonly set to 2 for the cubic spline kernel.

The standard SPH gradient and Laplacian approximations assume the following expressions, respectively [5]:

$$\nabla f(\mathbf{r}_i) = \langle \nabla f \rangle_i = \sum_{j \in \mathcal{S}(i)} V_j (f_j - f_i) \nabla_i W_{ij} \tag{3.96}$$

$$\nabla^2 f(\mathbf{r}_i) = \langle \nabla^2 f \rangle_i = 2 \sum_{j \in \mathcal{S}(i)} V_j (\mathbf{e}_{ij} \cdot \nabla_i W_{ij}) \frac{f_i - f_j}{|\mathbf{r}_{ij}|} \ , \tag{3.97}$$

where $\mathbf{e}_{ij} = \dfrac{\mathbf{r}_{ij}}{|\mathbf{r}_{ij}|}$ and $\nabla_i$ denotes differentiation in space with respect to the coordinates of SPH particle $i$; i.e.,

$$\nabla_i W_{ij} = \frac{\mathbf{r}_{ij}}{|\mathbf{r}_{ij}|} \frac{\partial W}{\partial q} \frac{\partial q}{\partial |\mathbf{r}_{ij}|}\bigg|_{i,j} = \frac{-15 \mathbf{r}_{ij}}{14\pi h^5 q} \times \begin{cases} (2-q)^2 - 4(1-q)^2, & 0 \le q < 1 \\ (2-q)^2, & 1 \le q < 2 \\ 0, & q \ge 2 \end{cases} . \tag{3.98}$$

The outcome of the SPH discretization steps described above can be conveniently represented in matrix form. To that end, at the beginning of a time step one computes and



Draft
Milad Rakhsha

stores the discretization matrices $\mathbf{A}^G$ and $\mathbf{A}^L$ that arise from either the standard discretization Eqs. (3.96)-(3.97). For instance, working with Eq. (3.97), $\langle \nabla^2 f \rangle_i$ can be expressed as follows:

$$\langle \nabla^2 f \rangle_i = \mathbf{A}_i^L \mathbf{f}, \tag{3.99}$$

$$\mathbf{f} = \begin{bmatrix} f_1 & f_2 & \cdots & f_P \end{bmatrix}^T \tag{3.100}$$

$$\mathbf{A}_i^L = \Big[ \cdots, \underbrace{2\textstyle\sum_{j\in\mathcal{S}(i)} V_j (\mathbf{e}_{ij} \cdot \nabla_i W_{ij}) \frac{1}{|\mathbf{r}_{ij}|}}_{i^{th} \text{ element}}, \cdots, \underbrace{-2V_j (\mathbf{e}_{ij} \cdot \nabla_i W_{ij}) \frac{1}{|\mathbf{r}_{ij}|}}_{j^{th} \text{ element s.t. } j\in\mathcal{S}(i)}, \cdots \Big] \tag{3.101}$$

where the subscript $P$ denotes the number of SPH particles in the domain. Similarly, the gradient of a scalar field $\langle \nabla f \rangle_i$ and divergence of a vector field $\langle \nabla . \mathbf{u} \rangle_i$ may be computed from Eq. (3.96) as follows:

$$\langle \nabla f \rangle_i = \begin{bmatrix} \mathbf{A}_i^{Gx} \\ \mathbf{A}_i^{Gy} \\ \mathbf{A}_i^{Gz} \end{bmatrix} \mathbf{f}, \tag{3.102}$$

$$\langle \nabla \cdot \mathbf{u} \rangle_i = \mathbf{A}_i^{Gx} \mathbf{u}_x + \mathbf{A}_i^{Gy} \mathbf{u}_y + \mathbf{A}_i^{Gz} \mathbf{u}_z, \tag{3.103}$$

$$\mathbf{f} = \begin{bmatrix} f_1, & f_2, & \cdots, & f_P \end{bmatrix}^T, \tag{3.104}$$

$$\mathbf{u}_x = \begin{bmatrix} (u_x)_1, & (u_x)_2, & \cdots, & (u_x)_P \end{bmatrix}^T, \tag{3.105}$$

$$\mathbf{u}_y = \begin{bmatrix} (u_y)_1, & (u_y)_2, & \cdots, & (u_y)_P \end{bmatrix}^T, \tag{3.106}$$

$$\mathbf{u}_z = \begin{bmatrix} (u_z)_1, & (u_z)_2, & \cdots, & (u_z)_P \end{bmatrix}^T, \tag{3.107}$$

where

$$\mathbf{A}_i^{Gx} = \Big[ \cdots, -\textstyle\sum_{j\in\mathcal{S}(i)} V_j \nabla_{i,1} W_{ij}, \cdots, V_j \nabla_{i,1} W_{ij}, \cdots \Big] \tag{3.108}$$

$$\mathbf{A}_i^{Gy} = \Big[ \cdots, -\textstyle\sum_{j\in\mathcal{S}(i)} V_j \nabla_{i,2} W_{ij}, \cdots, V_j \nabla_{i,2} W_{ij}, \cdots \Big] \tag{3.109}$$

$$\mathbf{A}_i^{Gz} = \Big[ \cdots, -\textstyle\sum_{j\in\mathcal{S}(i)} V_j \nabla_{i,3} W_{ij}, \cdots, V_j \nabla_{i,3} W_{ij}, \cdots \Big] \tag{3.110}$$

$$\mathbf{A}_i^{G} = \Big[ \cdots, \underbrace{-\textstyle\sum_{j\in\mathcal{S}(i)} V_j \nabla_i W_{ij}}_{i^{th} \text{ element}}, \cdots, \underbrace{V_j \nabla_i W_{ij}}_{j^{th} \text{ element s.t. } j\in\mathcal{S}(i)}, \cdots \Big]. \tag{3.111}$$



Draft
Milad Rakhsha

The system level matrices $\mathbf{A}^G$ and $\mathbf{A}^L$ are obtained as

$$\langle \nabla f \rangle^x = \begin{bmatrix} \langle \nabla f \rangle_1^x \\ \langle \nabla f \rangle_2^x \\ \vdots \\ \langle \nabla f \rangle_P^x \end{bmatrix} = \mathbf{A}^{Gx}\mathbf{f}, \quad \langle \nabla f \rangle^y = \begin{bmatrix} \langle \nabla f \rangle_1^y \\ \langle \nabla f \rangle_2^y \\ \vdots \\ \langle \nabla f \rangle_P^y \end{bmatrix} = \mathbf{A}^{Gy}\mathbf{f}, \quad \langle \nabla f \rangle^z = \begin{bmatrix} \langle \nabla f \rangle_1^z \\ \langle \nabla f \rangle_2^z \\ \vdots \\ \langle \nabla f \rangle_P^z \end{bmatrix} = \mathbf{A}^{Gz}\mathbf{f}, \tag{3.112}$$

$$\langle \nabla \cdot \mathbf{u} \rangle = \begin{bmatrix} \langle \nabla \cdot \mathbf{u} \rangle_1, & \langle \nabla \cdot \mathbf{u} \rangle_2, & \cdots, & \langle \nabla \cdot \mathbf{u} \rangle_P \end{bmatrix}^T = \mathbf{A}^{Gx}\mathbf{u}_x + \mathbf{A}^{Gy}\mathbf{u}_y + \mathbf{A}^{Gz}\mathbf{u}_z, \tag{3.113}$$

$$\mathbf{A}^{Gx} = \begin{bmatrix} \mathbf{A}_1^{Gx} \\ \mathbf{A}_2^{Gx} \\ \vdots \\ \mathbf{A}_P^{Gx} \end{bmatrix}, \quad \mathbf{A}^{Gy} = \begin{bmatrix} \mathbf{A}_1^{Gy} \\ \mathbf{A}_2^{Gy} \\ \vdots \\ \mathbf{A}_P^{Gy} \end{bmatrix}, \quad \mathbf{A}^{Gz} = \begin{bmatrix} \mathbf{A}_1^{Gz} \\ \mathbf{A}_2^{Gz} \\ \vdots \\ \mathbf{A}_P^{Gz} \end{bmatrix} \tag{3.114}$$

$$\langle \nabla^2 f \rangle = \mathbf{A}^L \mathbf{f}, \tag{3.115}$$

$$\langle \nabla^2 f \rangle = \begin{bmatrix} \langle \nabla^2 f \rangle_1, & \langle \nabla^2 f \rangle_2, & \cdots, & \langle \nabla^2 f \rangle_P \end{bmatrix}^T, \tag{3.116}$$

$$\mathbf{A}^L = \begin{bmatrix} \mathbf{A}_1^L \\ \mathbf{A}_2^L \\ \vdots \\ \mathbf{A}_P^L \end{bmatrix}. \tag{3.117}$$



Draft
Milad Rakhsha

**ME751-Assignment 9**

**Problem 1:** Obtain the *steady-state* solution of the advection diffusion equation in the 1D domain $x \in [0, 1]$ using the Finite-Difference method for the following boundary conditions $\phi(0) = 0$, and $\left.\dfrac{\partial \phi}{\partial x}\right|_{x=1} = 0$.

- Start with Eq. 3.9.
- In steady-state $\partial\phi/\partial t = 0$, and $\phi$ is only a function of space, i.e. $\phi = \phi(\boldsymbol{x})$.
- Assume that the velocity field in given by a constant function $u(x, t) = u(x) = 1$, the diffusion is also constant $D = D(x, t) = 1$ and $s = s(x, t) = 1$. It is easy to play around with the choice to these functions in your numerical solution to see different behaviors of the advection-diffusion equation. For instance, if $u(x, t) = 0$ and $s(x, t) = 0$ you are left with only a diffusion term that corresponds to the *Laplace* equation. If $u(x, t) = 0$ and $s(x, t) \neq 0$ then you are solving a *Poisson* equation.
- Using $n + 1$ points, discretize the domain into $n$ equidistant elements of size $h = 1/n$ as shown in Fig. 12.

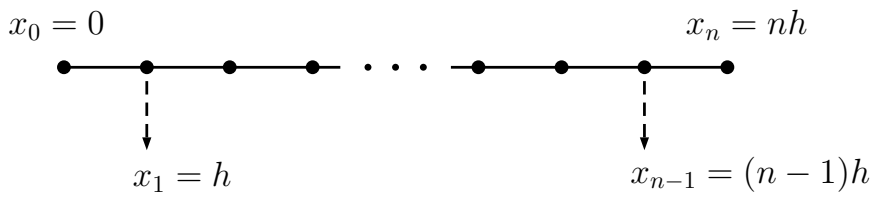



Figure 12: 1D space discretization of the domain $x \in [0, 1]$

- Note that the discrete solution of the $\phi = \phi(x)$ will reside on these $n+1$ discrete points, i.e. we are looking to find $\phi_1 = \phi(x_1)$, $\phi_2 = \phi(x_2)$, ..., and $\phi_{n-1} = \phi(x_{n-1})$.
- Note that the solution of the $\phi = \phi(x)$ is given at $\phi(x_0) = \phi(0) = 0$ through the Dirichlet boundary condition.
- Note that the solution of $\phi(x_n) = \phi(1)$ is not known but the Neumann boundary condition provides *some* information through the gradient of $\phi$. This should be incorporated into your code.
- With proper simplification, you can use Eq. 3.18 to form an equation for each internal node in the domain.
- The algebraic equation for the rightmost node can be derived from Eq. 3.20. The algebraic equation for the leftmost node is simply $\phi_0 = 0$



Draft
Milad Rakhsha

- Form and solve a linear system of form $\boldsymbol{A\phi} = \boldsymbol{b}$ from the $n+1$ equations you derived. Herein, $\boldsymbol{A}_{(n+1)\times(n+1)}$ is the coefficient matrix, $\boldsymbol{\phi} = \begin{bmatrix}\phi_0 & \phi_1 & \cdots & \phi_{n-1} & \phi_n\end{bmatrix}^T$ is the solution vector you are looking form.
- Plot the numerical solution of $\phi = \phi(x)$.
- Validate your solution; You can use edge cases to see the validity of your implementation. For instance, if $u = 0$, you can integrate twice and apply the boundary conditions to $-\nabla^2\phi = -\frac{\partial^2\phi}{\partial x^2} = s$. Another edge case is when only the advection term is present; in this case you have a linear equation $u\frac{\partial\phi}{\partial x} = s$. You can again integrate and apply the Dirichlet condition at $x = 0$. Note that in principal you can only apply one boundary condition in this scenario. In practice, removing the diffusion term creates numerical problems that could be explained in many different ways. Try to use a *compatible* Neumann boundary condition when there is no diffusion, i.e. impose $\frac{\partial\phi}{\partial x}\Big|_{x=1} = \frac{s}{u}\Big|_{x=1}$ to see if it helps.



Draft
Milad Rakhsha

**Problem 2:** Solve problem 1 using the Finite-Volume method.

- Discretize the domain into $n$ equidistant volumes $V_1$, $V_2$, ..., and $V_n$ as shown in Fig. 13.
- Note that the solution of the $\phi$ resides on these volumes. $\phi_1$, $\phi_2$, ..., $\phi_n$ represent the average value of $\phi$ in volumes $V_1$, $V_2$, ..., and $V_n$ respectively (see Eq. 3.28)

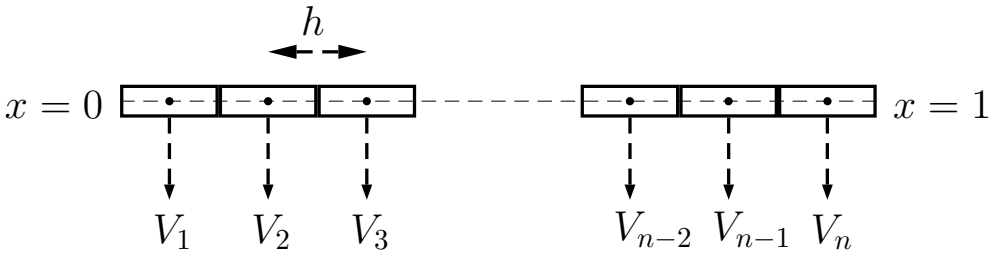



Figure 13: 1D space discretization of the domain $x \in [0, 1]$ into *finite volumes*

- Note that in 1D these volumes are simply **line segments**, although you may imagine 3D volumes with the cross section area of $A$, if it helps you in visualizing the problem.
- Form an algebraic equation for these volumes according to Eq. 3.34. Use central differencing for the advection term.
- Incorporate the boundary conditions into the flux terms according to section §3.3.4.
- Form and solve a linear system of form $\boldsymbol{A\phi} = \boldsymbol{b}$ from the $n$ equations you derived for the $n$ volumes. Herein, $\boldsymbol{A}_{n\times n}$ is the coefficient matrix, $\boldsymbol{\phi} = \begin{bmatrix}\phi_1 & \phi_2 & \cdots & \phi_{n-1} & \phi_n\end{bmatrix}^T$ is the solution vector you are looking form.
- Plot the numerical solution of $\phi = \phi(x)$.
- (Optional) Try to increase the constant velocity field and see if it creates any problem in the condition of the matrix $\boldsymbol{A}$ and the solution quality. Use the upwind differencing scheme in Eq. 3.39 for advective flux and see if it helps.



Draft
Milad Rakhsha

**ME751-Assignment 10**

**Problem 1:** Solve problem 1 using the Finite-Element method.

- Using $n+1$ points, discretize the domain into $n$ equidistant elements of size $h = 1/n$ as shown in Fig. 14.

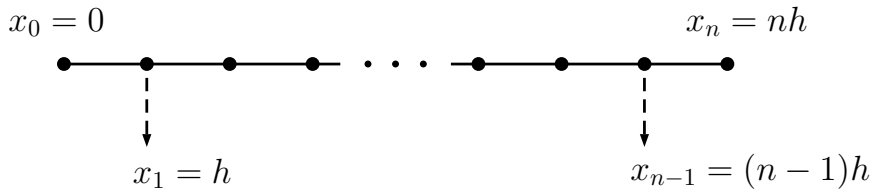



Figure 14: 1D space discretization of the domain $x \in [0, 1]$

- Note that the discrete solution of the $\phi = \phi(x)$ will reside on these $n+1$ discrete points, i.e., we are looking to find $\phi_1 = \phi(x_1)$, $\phi_2 = \phi(x_2)$, ..., and $\phi_{n-1} = \phi(x_{n-1})$.
- We aim at solving a simplified form of Eq. 3.61 or Eq. 3.60, knowing the problem is steady-state.
- Use linear interpolation functions as shown in Fig. 9. Use Eqs. 3.65-3.66 along with Eq. 3.68 to form elements' stiffness matrices and the load vector, respectively.
- Calculate the elements' matrices and vectors *exactly* is straightforward in this problem. For instance, calculating analytically the advection matrix for an element $e_k$ requires calculating the following integral

$$\boldsymbol{C}_{e_k} = \int_{e_k} \begin{bmatrix} \psi_1 \nabla \psi_1 & \psi_1 \nabla \psi_2 \\ \psi_2 \nabla \psi_1 & \psi_2 \nabla \psi_1 \end{bmatrix} d\boldsymbol{x},$$

where linear shape functions for a general element $k$ take the following form

$$\psi_1(x) = \frac{x_{k+1} - x}{x_{k+1} - x_k}$$
$$\psi_2(x) = \frac{x - x_k}{x_{k+1} - x_k}.$$

In this 1D problem, $k$ and $k+1$ are the global indices of nodes comprising the element $k$ (or respectively 1 and 2 in the element local indexing) Note that $\psi_1(x_k) = 1$, $\psi_1(x_{k+1}) = 0$, $\psi_2(x_{k+1}) = 1$, and $\psi_2(x_k) = 0$ satisfy properties of the shape functions mentioned in section §3.4. If you like to practice numerical integration of these matrices you can follow section §3.4.5



Draft
Milad Rakhsha

- build the global stiffness matrix and load vector from the local matrices you just built (see Fig. 10).
- Form and solve a linear system of Eq. 3.61. Note that $\boldsymbol{\phi} = \begin{bmatrix} \phi_0 & \phi_1 & \cdots & \phi_{n-1} & \phi_n \end{bmatrix}^T$ is the solution vector you are looking form.
- Plot the numerical solution of $\phi = \phi(x)$.



Draft
Milad Rakhsha

**Problem 2:** Solve problem 1 using the Smoothed Particle Hydrodynamics method.

- Using $n+1+2+2 = n+5$ points, discretize the domain into $n+4$ equidistant segments of size $h = 1/n$ as shown in Fig. 15. Note that we add 2 *ghost* markers to the left and 2 to the right of the domain to ensure enough markers exist in the support domain of markers at $x_2 = 0$ and $x_n = nh$

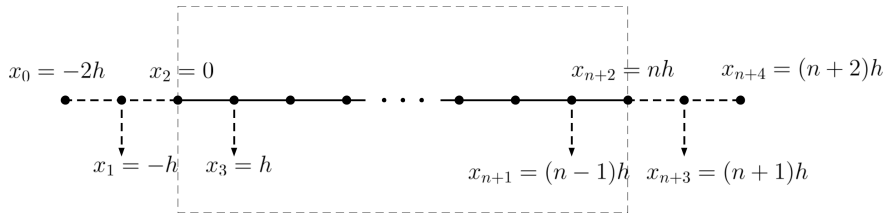



Figure 15: 1D space discretization of the domain $x \in [0, 1]$ in SPH method

- Use the following kernel in 1D:

$$W(|\mathbf{r}|, h) = \frac{1}{6h} \times \begin{cases} (2-q)^3 - 4(1-q)^3, & 0 \le q < 1 \\ (2-q)^3, & 1 \le q < 2 \\ 0, & q \ge 2 \end{cases}, \tag{3.118}$$

  the kernel gradient in 1D is:

$$\nabla_i W_{ij} = \frac{\mathbf{r}_{ij}}{|\mathbf{r}_{ij}|} \frac{\partial W}{\partial q} \frac{\partial q}{\partial |\mathbf{r}_{ij}|}\bigg|_{i,j} = \frac{-3\mathbf{e}_{ij}}{6h^2} \times \begin{cases} (2-q)^2 - 4(1-q)^2, & 0 \le q < 1 \\ (2-q)^2, & 1 \le q < 2 \\ 0, & q \ge 2 \end{cases}. \tag{3.119}$$

- How can you manipulate the algebraic equation for node $x_2$ to incorporate the Dirichlet boundary condition? See Eq. 3.94 and note the symmetry between $x_1$ and $x_3$ and also between $x_0$ and $x_4$ w.r.t the $x_2 = 0$.
- How can you manipulate the algebraic equation for node $x_{n+2}$ to incorporate the Dirichlet boundary condition? See Eq. 3.96 and note the symmetry between $x_{n+3}$ and $x_{n+1}$ and also between $x_{n+4}$ and $x_n$ w.r.t the $x_{n+2}$. This is more involved for a general 3D problem but is easy for this problem where $\frac{\partial \phi}{\partial x}\Big|_{x=1} = 0$.
- Use Eqs. 3.96 and 3.97 to discretize the advection and diffusion terms.
- Form and solve a linear system of form $\boldsymbol{A}\boldsymbol{\phi} = \boldsymbol{b}$ from the $n+5$ equations you derived. Herein, $\boldsymbol{A}_{(n+5)\times(n+5)}$ is the coefficient matrix, $\boldsymbol{\phi} = \begin{bmatrix} \phi_0 & \phi_1 & \phi_2 & \cdots & \phi_n & \phi_{n+3} & \phi_{n+4} \end{bmatrix}^T$ is the solution vector you are looking form.
- Plot the numerical solution of $\phi = \phi(x)$.



Draft
Milad Rakhsha

**Problem 5 :** Solve the unsteady form of the advection-diffusion equation over $t \in [0, 1]$ using Finite-Difference and assumption made in Problem 1.

- Use forward Euler method in Eq. 3.15 for time-discretization. Note that each time step of the solution involves all the calculations required in problem 1.

**Problem 6 :** Solve the unsteady form of the advection-diffusion equation over $t \in [0, 1]$ using Finite-Volume and assumption made in Problem 2.

- Use forward Euler method in Eq. 3.35 for time-discretization. Note that each time step of the solution involves all the calculations required in problem 2, while you need to keep track of previous solution vector.
- Use backward Euler method in Eq. 3.36 for time-discretization. Note that each time step of the solution involves all the calculations required in problem 2, while you need to keep track of 2 previous solution vectors.
- Try to increase the time-step and see if the solution diverges.

**Problem 7:** Solve the unsteady form of the advection-diffusion equation over $t \in [0, 1]$ using Finite element method, assumption made in Problem 3 and integration methods discussed in Problem 5.



Draft
Milad Rakhsha

# 4 Numerical Solution of Fluid Flows

In this section we explain the numerical solution of the fluid flow problems. The underlying equations of motion are the mass (Eq. 2.18) and momentum balance (Eq. 2.38) as described in Section §2.3. The Lagrangian form of the equations for incompressible flow ($D\rho/Dt = 0$) and Newtonian fluids ($\boldsymbol{\sigma}_{dev} = 2\mu\boldsymbol{E} = \mu(\nabla\mathbf{u} + \nabla\mathbf{u}^{\mathbf{T}})$) may be simplified to the following:

$$\frac{D\rho}{Dt} + \rho\nabla \cdot \boldsymbol{u} = 0, \tag{4.1a}$$

$$\frac{D\boldsymbol{u}}{Dt} = -\frac{1}{\rho}\nabla p + \nu\nabla^2\boldsymbol{u} + \boldsymbol{f}_b, \tag{4.1b}$$

where $\nu = \mu/\rho$ and $\rho$ are the fluid kinematic viscosity and density, respectively; $\boldsymbol{f}_b$ is the volumetric force density (source term); and $\boldsymbol{u}$ and $p$ are the flow velocity and pressure, respectively.

It is important to notice that the pressure does not appear directly in the equations, but as $\nabla p$. With the incompressible flow assumption, pressure is also independent of $\rho$, and is solely a *mechanical* property (average of normal stresses), as opposed to being a *thermodynamical* property in the compressible flows. In what follows we describe the Chorin's projection method and how the Poisson equation for pressure is formed.

## 4.1 Helmholtz Decomposition

This algorithm is based on the Helmholtz-Hodge decomposition, which states that any vector field $\boldsymbol{u}$ may be decomposed into a solenoidal ($\boldsymbol{u}_{sol}$) and an irrotational ($\boldsymbol{u}_{irrot}$) part as follows:

$$\boldsymbol{u} = \boldsymbol{u}_{irrot} + \boldsymbol{u}_{sol}, The \tag{4.2}$$

where the solenoidal part is divergent-free ($\nabla.\boldsymbol{u}_{sol} = 0$). The irrotational part is curl-free ($\nabla \times \boldsymbol{u}_{irrot=0}$) and can be written as the gradient of a potential/scalar field ($\boldsymbol{u}_{irrot} = \nabla\phi$). Interestingly, the irrotational part is a conservative vector field, and the line integral of this field is path-independent. By taking divergence of $\boldsymbol{u}$ and using the divergent-free property of $\boldsymbol{u}_{sol}$, it can be seen that

$$\nabla \cdot \boldsymbol{u} = \nabla \cdot \boldsymbol{u}_{irrot} = \nabla^2\phi, \tag{4.3}$$

and given that $\boldsymbol{u}$ is known, the Poisson equation 4.3 can be solved for $\phi$ which finally results in $\boldsymbol{u}_{sol} = \boldsymbol{u} - \boldsymbol{u}_{irrot} = \boldsymbol{u} - \nabla\phi$.

In a similar fashion, we can decompose the force field per unit mass ($\frac{D\boldsymbol{u}}{Dt}$) of Equation 4.1b into a solenoidal field ($\nu\nabla^2\boldsymbol{u} + \boldsymbol{f}_b$) and an irrotational field ($-\frac{1}{\rho}\nabla p$). The semi-discrete form (discretized only in time) of Eq. 4.1b is written as follows:

$$\frac{(\boldsymbol{u}^{n+1} - \boldsymbol{u}^n)}{\Delta t} = \frac{(\boldsymbol{u}^{n+1} - \boldsymbol{u}^*)}{\Delta t} + \frac{(\boldsymbol{u}^* - \boldsymbol{u})}{\Delta t} = -\frac{1}{\rho}\nabla p + \nu\nabla^2\boldsymbol{u}^n + \boldsymbol{g} \tag{4.4}$$



Draft
Milad Rakhsha

## 4.2 Non-Incremental Projection Method

Based on the previous discussion we do the following decomposition:

$$\frac{(\boldsymbol{u}^* - \boldsymbol{u}^n)}{\Delta t} = +\nu\nabla^2\boldsymbol{u}^n + \boldsymbol{g} = \boldsymbol{f}^n \quad x \in \Omega, \qquad \boldsymbol{u}^* = 0 \quad x \in \partial\Omega \tag{4.5}$$

$$\frac{(\boldsymbol{u}^{n+1} - \boldsymbol{u}^*)}{\Delta t} = -\frac{1}{\rho}\nabla p^{n+1} \quad x \in \Omega, \qquad \nabla.\boldsymbol{u}^{n+1} = 0 \quad \in \partial\Omega. \tag{4.6}$$

Eq. 4.5 is the predictor step and it can be used to find the intermediate velocity $\boldsymbol{u}^*$:

$$\boldsymbol{u}^* = \boldsymbol{f}^n\Delta t + \boldsymbol{u}^n. \tag{4.7}$$

Moreover, if pressure is known, Eq. 4.6 may be used to find $\boldsymbol{u}^{n+1}$ as follows

$$\boldsymbol{u}^{n+1} = -\Delta t\frac{1}{\rho}\nabla p^{n+1} + \boldsymbol{u}^*. \tag{4.8}$$

In order to obtain the pressure equation, using the approach led to Eq. 4.3, we take divergence of the Eq. 4.6 and obtain the Poisson equation for pressure.

$$\frac{\nabla \cdot \boldsymbol{u}^{n+1} - \nabla \cdot \boldsymbol{u}^*}{\Delta t} = -\frac{1}{\rho}\nabla^2 p^{n+1} \tag{4.9}$$

Note that from Continuity (Eq. 4.1a) with incompressible flow assumption $\frac{d\rho}{dt} = 0$ we can write $\nabla.\boldsymbol{u}^{n+1} = 0$ and simplify Eq. 4.9 to

$$\frac{1}{\rho}\nabla^2 p^{n+1} = \frac{\nabla \cdot \boldsymbol{u}^*}{\Delta t}, \quad \nabla p^{n+1} \cdot \boldsymbol{n}|_{\partial\Omega} = 0 \tag{4.10}$$

Next, Eq. 4.8 is used to update the velocities. The algorithm described above, *velocity-based projection*, is usually preferred when density variation is small and $\frac{d\rho}{dt} = 0$ is hold. For Lagrangian methods such as SPH, when working with free-surface flows, the *density-based projection* method is considered to be more accurate. In this method continuity equation is used to replace the velocity divergence term $\frac{\nabla.\boldsymbol{u}^*}{\Delta t}$ in Eq. 4.12. The semi-discrete continuity equation is as follows

$$\frac{\rho^* - \rho^n}{\Delta t} = -\rho^n\nabla \cdot \boldsymbol{u}^*. \tag{4.11}$$

Using the right hand side of 4.11, one may write Eq. 4.12 as follows:

$$\frac{1}{\rho}\nabla^2 p^{n+1} = -\frac{1}{\rho^n}\frac{\rho^* - \rho^n}{\Delta t^2}, \tag{4.12}$$

which takes into consideration the density variation as a source term in the Poisson equation.



Draft
Milad Rakhsha

Another important aspect of the discretization of Eq. 4.5 has to do with the treatment of the viscous forces. This discretization may be chosen to be explicit, implicit or semi-implicit in terms of $\boldsymbol{u}^*$ as follows:

$$\frac{(\boldsymbol{u}^* - \boldsymbol{u}^n)}{\Delta t} = +\nu(\theta\nabla^2\boldsymbol{u}^* + (1-\theta)\nabla^2\boldsymbol{u}^n) + \boldsymbol{g} = \boldsymbol{f}^n, \tag{4.13}$$

Eq. 4.14 corresponds to the fully-explicit scheme (Eq 4.5) for $\theta = 0$, and to fully-implicit scheme for $\theta = 1$. Crank-Nicolson discretization is obtained via $\theta = 0.5$.

## 4.3 Incremental Projection Method

The incremental projection method with the Crank-Nicolson correction applied to the prediction step is as follows:

$$\frac{(\boldsymbol{u}^* - \boldsymbol{u}^n)}{\Delta t} = \frac{\nu}{2}(\nabla^2\boldsymbol{u}^* + \nabla^2\boldsymbol{u}^n) - \frac{1}{\rho}\nabla^2 p + \boldsymbol{g} = \boldsymbol{f}^n. \tag{4.14}$$

$$\frac{(\boldsymbol{u}^{n+1} - \boldsymbol{u}^*)}{\Delta t} = -\frac{1}{\rho}(\nabla p^{n+1} - \nabla p^{n+1}) = -\frac{1}{\rho}(\nabla\phi), \tag{4.15}$$

$$\phi = p^{n+1} - p^n \tag{4.16}$$

Herein, the Poisson equation is solved for $\phi$ instead of $p^{n+1}$. In Eulerian grid, the pressure correction is performed via $p^{n+1} = p^n + \phi$, whereas in the Lagrangian methods the pressure correction is as follows:

$$p^{n+1} = p^n + \phi + \nabla p^{n+1}.(\boldsymbol{x}^{n+1} - \boldsymbol{x}^n) \tag{4.17}$$

See [6, 7] for more details about the advantages of the incremental projection method over the non-incremental variation.

In general, any of the space discretization techniques discussed in sections §3.2-§3.5 can be applied to solve these equations in a discrete sense.



Draft
Milad Rakhsha

# References


[1] Dmitri Kuzmin. A guide to numerical methods for transport equations. *University Erlangen-Nuremberg*, page 24, 2010.

[2] Susanne Brenner and Ridgway Scott. *The mathematical theory of finite element methods*, volume 15. Springer Science & Business Media, 2007.

[3] OpenFOAM Foundation. OpenFOAM: An open source free CFD code. http://www.openfoam.org.

[4] CFD Open. Openfoam programmer's guide. *OpenFOAM Foundation*, 2(0), 2011.

[5] J. J. Monaghan. Smoothed Particle Hydrodynamics. *Reports on Progress in Physics*, 68(1):1703–1759, 2005.

[6] N. Trask, M. Maxey, K. Kimb, M. Perego, M. L. Parks, K. Yang, and J. Xu. A scalable consistent second-order SPH solver for unsteady low Reynolds number flows. *Comput. Methods Appl. Mech. Engrg.*, 289:155–178, 2015.

[7] S Majid Hosseini and James J Feng. Pressure boundary conditions for computing incompressible flows with SPH. *Journal of Computational Physics*, 230(19):7473–7487, 2011.




Draft
Milad Rakhsha